
AI Grand Prix Physical Qualifier

Onboarding Crash Course & Engineering Journal

Team Electric Fire (Team 10) · repo `Code-Red-Cables/AI_GP`, branch `isaacsim @ 66b61c5`

Revision 1: 16 September 2026, the evening before the team arrived at LC3

Revision 2: 17 September 2026, written on site with the real drone and the specification in hand

How to read this document

Part I (Sections 1–5) is a crash course: what the competition is, the minimum drone-autonomy vocabulary, how our system is put together, where things live in the repo, and what the physical course looks like. **Part II (Sections 6–12)** is the engineering journal: project history, the Isaac PPO stack in depth, audit findings with evidence, the durability (stress) study run on 16 Sep, Plan B, and next steps.

Evidence tags: ^[V] verified by reading code, running it, or checking git on 16 Sep; ^[I] inference; ^[ext] from a public source outside our repo (organizer site, the Betaflight source, or the public repository of AI of Sauron, another PQ team). The `AI_GP` repository was treated as *read-only*; every experiment ran on scratch copies.

People and plans named in this document.

- **Gene** (Geneustace Wicaksono) is the team's main developer. He built and trained the simulator pilot described here.
- **Plan A:** fly Gene's trained neural network at racing speed.
- **Plan B:** fly a slower, hand-written step-by-step controller that stops and lines up in front of each gate (Section 10.2).
- **AI of Sauron:** another physical-qualifier team, from Georgia Tech, whose public repository describes the same drone.
- **Where the files are:** every file this document mentions (the corrections patch, the test tools, the one-hour retrained model, the raw results) is in the `AI_GP` repository on the branch `pq-analysis-0916`, under `analysis/2026-09-16/`. The retrained model is at `isaac_drone_racer/models/pq_retrain_1h_corrected.pt`. All experiments ran on bojro's laptop (RTX 4060 laptop GPU, Windows with WSL).

Summary

1. **The course is fixed and known.** Ten numbered gates (gate 9 is a stacked double gate, gate 6 is a split-S), flown in order, two laps. Only small as-built differences from the published map are expected.
2. **What we had for the physical qualifier (PQ):** a neural-network pilot that Gene trained by reinforcement learning (trial and error in a simulator, explained in Section 3.2). It is saved as the file `pq_speed_best.pt` and is called “Gene’s model” throughout this document. In simulation it lapped in **7.35–7.47 s**, averaging 14.4 m/s and peaking at 25.4 m/s (verified headless on 16 Sep). **As of 17 September this model is abandoned** — it does not survive on the corrected simulator. It is kept in this document as evidence of *how* a policy can look excellent and be worthless, which is the most useful thing it now does.
3. **Nothing in the repo talks to the real drone yet.** The team’s flight program (the Python “client” in the repository root) speaks MAVLink, the protocol of the virtual-qualifier simulator; the real drone is Betaflight + MSP over UART to a Jetson. Camera capture, a real-image gate detector and an onboard gate counter are also still to be built.
4. **Four corrections for the team:** (a) a bug in the training code stops it from counting gate passes, so any further training fails (proven by a side-by-side test; the fix is one line); (b) the simulated camera is 90° wide, the real one is 75°; (c) Gene’s simulated course is about 93 % of the true size (it must be scaled up by about 7.5 %), and its gates sit 0.65 m too high; (d) the real flight controller turns stick commands into rotation speed and thrust along curves, but the simulator assumes straight-line relationships.
5. **Durability:** the policy is fast but brittle.
 - **Real camera:** 42 crashes per 100 gates.
 - **True course size:** 10 per 100.
 - **Both together:** it clears 4 gates and crashes, every time.
 - **After a one-hour retrain on the corrected simulator:** 10× fewer crashes under expected conditions, but still only 6–7 % two-lap success. A real retrain takes hours (Sections 9–9.3).
6. **Strategy:** scoring is most gates first, time second, best run of Mon 21–Tue 22 Sep. Bank a valid run with Plan B first, then attempt speed.
7. **Plan B tested in simulation** (Section 10.2), using the “balanced” setting: gates at 2.5 m/s, cruise 4 m/s. It finished **100 %** of runs under expected conditions and 88 % under rougher ones, in about **2:05** for two laps.
 - “Safe” takes 2:21 (93 % under rougher conditions); “fast” takes 1:52 (79 % under rougher conditions).
 - A barometer is optional.
 - **Camera tilt must be measured:** an unknown 5° error is its biggest single risk.

Contents

Status at 17 September — read this before anything else	3
Part I — Crash course	6
1 The competition in two pages	6
1.1 What the AI Grand Prix is	6
1.2 PQ rules that matter	6
1.3 The hardware you will fly	7
1.4 Is the course the same as in our simulator? (FAQ)	7
2 Drone-racing autonomy in ten minutes	7
2.1 The loop	7
2.2 Vocabulary	7
3 How our system works	9
3.1 Big picture	9
3.2 Reinforcement learning in plain terms	10
3.3 What the policy sees and does	10
3.4 How it was trained	11
4 Repo tour	12
4.1 Running things on the team laptop (WSL)	12
5 The physical course	13
Part II — Engineering journal	15
6 Project history	15
7 Isaac training history and Gene’s best model	15
8 Audit findings	16
8.1 Gate-counting bug in the current Isaac code	17
8.2 Camera field of view: who is right?	17
8.3 Track size and gate heights: who is right?	18
8.4 Flight-controller configuration	18
8.5 The missing deployment layer	19
8.6 Smaller items	19
9 Durability study (16 Sep)	19
9.1 Method	19
9.2 Results	20
9.3 Retraining on the corrected simulator (16 Sep)	22
9.4 How long proper retraining takes on bojro’s laptop	24

10 Plan A and Plan B	25
10.1 Plan B speed budget	25
10.2 Plan B tested in simulation (16 Sep)	26
11 Knowing which gate is which	28
11.1 The problem, and why it is not a perception problem	28
11.2 What the policy actually reads	28
11.3 “The model will learn the order from simulation”	29
11.4 What we build instead	30
11.5 The pipeline	30
11.6 Evidence	31
12 Technical gap register	31
12.1 First, what Betaflight actually is	31
12.2 Gap register	32
13 The plan, organised as parallel workstreams	35
14 Field procedures	35
14.1 P1 — Camera calibration	35
14.2 P2 — Camera tilt	36
14.3 P3 — Vehicle diagnostics: one battery, one log	36
14.4 P4 — Latency	36
14.5 P5 — Course survey	37
14.6 P6 — Gate detection and data labelling	37
15 Schedule and decision gates	39
16 Questions still open for the organizers	39
A Laptop environment (installed 16 Sep)	40
B Sources	40

Status at 17 September — read this before anything else

Revision 1 of this document was written on the evening of 16 September, off site, before anyone had touched the drone. The team is now at LC3, the race drone has been weighed, and the physical-qualifier specification (VADR-TS-005, issue 00.02) has been re-read line by line. Some of what follows in Parts I and II has been overtaken. **Where this section disagrees with a later section, this section is right.**

What changed

Item	Revision 1 said	Now
Gene's model	The main line for Plan A	Abandoned. It does not survive on the corrected simulator. Gene has reviewed the findings and agrees. The objective is now to train a new policy on the corrected simulator
Drone mass	0.6076 kg (the simulated value)	1745 g measured, race configuration. See the box below for why this matters less, and differently, than it looks
Camera	$f_x \approx 424 \text{ px}$ (74.1°), from another team's calibration	$f_x = f_y = 417.0 \text{ px}$ (75°), from our own specification. Ours must still be calibrated — the spec says no calibration matrices will be provided
Deployment layer	"Also still to be built"	Still the critical path, but much smaller than feared : the organizers ship a complete, working MSP layer on the drone (<code>~/target/msp/</code>), including a fake flight controller for offline development. Ours is the adapter, not the protocol
Link speed	Assumed our 60 Hz policy rate was achievable	The organizers state twice that the link gives "attitude at 30–50 Hz; a hard real-time control loop is not" realistic. May force a retrain at a lower policy rate
Camera mode	Assumed 1920×1080 at 60 fps	The carrier wires only two of four camera data lanes. That mode may not exist — and the 75° figure is quoted <i>for</i> it
"Next gate" input	Not called out	No source exists on the real drone. The simulator supplied it
Safety pilot	Assumed possibly available during a run	A valid run allows no human pilot intervention at all (spec §3.3). The override switch aborts a run; it cannot rescue one
Track footprint	$25.9 \times 50.25 \text{ m}$ from the drawing's ruler	Resolved. The official gate-coordinate reference gives $85 \times 165 \text{ ft} = 25.91 \times 50.29 \text{ m}$, confirming the drawing. The spec's " $60 \times 21 \text{ m}$ " is wrong. Our corrected track matches the official table exactly
Compute machine	Gene's machine	This laptop is now the primary simulation and test machine , with cloud GPU rented for training

What is still true

The durability study (Section 9) stands, and its ranking is the most useful thing in this document: **command delay and thrust calibration dominate every other error source**, with camera field of view and camera tilt next. The gate-counting bug analysis (Section 8.1) stands and the fix is applied. The map-audit method stands. Plan B's logic and speed budget stand, with the caveat that its numbers come from a purpose-built simulator, not from Isaac.

The weight finding, stated carefully — this one is easy to get wrong

The real drone is 1745 g; the simulated one is 607.6 g, an 8-inch airframe against a 5-inch one. The obvious response — “change the mass in the simulator and retrain” — is **almost entirely wasted effort**, and doing it carelessly makes things worse.

The simulator converts a throttle command into a force by multiplying by the drone’s own mass: $\text{thrust} = (\text{stick} / \text{hover_stick}) \times \text{mass} \times g$. Acceleration is force divided by mass, so the mass cancels out. Change only the mass and the simulated drone flies identically.

Three things do *not* cancel, and these are the real issues:

1. **Thrust-to-weight.** The simulator assumes the drone hovers at 25.5% throttle, which implies it can produce 3.5 times its own weight in thrust. That is a 600 g racing quad’s figure. A 1745 g machine carrying a Jetson more likely hovers near 35–45%, i.e. about 2.0–2.6 times its weight. If so, the simulator has been promising the policy acceleration the drone does not have. **Measuring this takes five minutes** (hover, read the throttle) and it is the single most valuable number available today. For scale: a 10% thrust error on its own took crashes from about 2 to **58 per 100 gates** in the durability study.
2. **Rotational inertia.** The real airframe resists being rotated roughly **11–16 times** more than the simulated one ($2.9\times$ the mass, with arms about $2.2\times$ longer, and inertia grows with the square of the arm). **Danger:** the simulator’s rate loop uses a fixed gain $\text{rate_kp} = 0.08$. If someone sets the real inertia and leaves that gain alone, the simulated drone becomes about $13\times$ more sluggish than anything real, and every policy trained on it is worthless. That gain is not physics — it stands in for Betaflight’s own controller, which is retuned for whatever airframe it is on. It must be re-derived from a measured response: $\text{gain} \approx \text{inertia} / \text{measured response time}$.
3. **Air drag**, which the simulator does not model at all.

The two gaps that block everything

1. **Our flight client cannot talk to the real drone.** Searching the entire flight client for “MSP” returns nothing; `controller.py` speaks MAVLink to the virtual qualifier’s simulator, which the real flight controller does not understand. **Much mitigated on 17 September:** the organizers ship a complete MSP layer on the board (\$), so what remains is an adapter, the Betaflight curve inversion and the sign/axis mapping — hours rather than days, and testable on a laptop against their fake flight controller. Still the critical path.
2. **Nothing tells the drone which gate is next.** The policy’s input includes a list of 18 on/off flags meaning “this is the gate you are going for”. In the simulator that came free, from the simulator. On the real course we must count gates ourselves from the camera — and the double gate counts twice per lap, which is exactly where a counter loses its place.

New on 17 September: the link may be slower than our policy

The organizers’ software guide states, twice and emphatically, that the flight-controller link is *telemetry-grade*: “MSP is request/response over a 115200-baud serial line. You poll; the FC answers. Attitude at 30–50 Hz is realistic; a hard real-time control loop is not.”

Our policy runs at **60 Hz** and takes gyro readings as part of its input. Sending stick commands at 60 Hz is fine — that is exactly what a radio link does, and Betaflight keeps the fast stabilisation loop internally, which is where it belongs. The problem is the *feedback* half: if gyro only arrives at 30–50 Hz, the simulator has been training the policy against a sensor faster than the one it will fly with.

Action: measure the achievable rate on day one with the organizers’ own `companion\_listener\_msp.py`, then set the simulator’s decimation to match *before* committing to a long training run. At 120 Hz physics, decimation 2 gives 60 Hz, 3 gives 40 Hz and 4 gives 30 Hz.

What the organizers already provide

Everything below is pre-installed in `~/target/` on the Jetson. No pip, no network, no root. Log in over a USB cable: `ssh dc1@192.168.55.1`, password `dc1`.

- `msp/msp.py` — MSPLink: all MSP framing, plus `attitude()`, `raw\_imu()`, `analog()`, `rc\_channels()`, `motors()`, `status()` and arming-blocker decoding. Thread-safe, background receive loop, counts corrupt

frames rather than crashing on them.

- `msp/msp\_rc.py` — **RCTransmitter**: streams stick commands at a fixed rate from a background thread **with a watchdog already built in**. Stop calling `set\_control()` and it falls back to an idle frame by itself. `arm()` refuses unless throttle is at minimum; `close()` disarms cleanly.
- `msp/fake\_fc.py` — **simulates a Betaflight flight controller on a laptop**, with deliberate fault modes (frozen, saturated, wrong scale, no accelerometer, bus errors). The whole test suite runs with nothing plugged in.
- `msp/setup\_jetson\_uart.sh` — frees the serial port from Linux’s login console. **Run once per board before anything else**; skipping it looks exactly like a dead flight controller.
- `msp\_bench.py` (info / telemetry / rc / demo), `imu\_check.py` (a real functional IMU test, not just “did it reply”), `companion\_listener\_msp.py` (live telemetry, and it reports the achievable poll rate).
- Camera: `live-view.py`, `live-view-pts.py` (hardware capture timestamps), `live-view-imu.py`, `show-camera.py`, `raw-view.py`, `frame-timestamps.py`, plus `bringup-check.sh` and `camera-bind-check.sh`.

The guide says it plainly: “**do not re-implement MSP framing.**”

First action for whoever reaches a board: `scp -r dcl0192.168.55.1:~/target ./`, and share it — that is what lets the rest of the team write and test adapter code without the drone.

Three constraints from those guides that will otherwise cost an afternoon each

1. **Only two of the camera’s four data lanes are wired** on this carrier, capping resolution $\times$ framerate. Run `v4l2-ctl -d /dev/video0 -l -l-list-formats-ext` and believe that list, not the datasheet. The 75° field of view is quoted for 1920 $\times$ 1080 at 60 fps.
2. **The image processor needs a graphics context.** Colour, auto-exposure and white balance run in NVIDIA’s ISP, which fails over a plain SSH session and falls back to a flat, dim software debayer. That is documented behaviour, not a broken camera.
3. **No deep-learning framework is installed.** OpenCV (GStreamer-enabled), NumPy, pyserial and python3-gi — no PyTorch. This makes the NumPy policy runtime *mandatory*. And never `pip install opencv-python`: it silently replaces the GStreamer build and the camera pipeline then fails with no error text.

There is also **no hardware sync wire between the camera and the flight controller**. Their guide calls its camera-IMU timing section “the section most likely to save you a week”. It is right; read it before building anything that fuses the two.

Where the working documents are

This PDF is the onboarding and history document. The live operational plan is kept as Markdown in the workspace root, because it changes daily: `FIELD\_PLAN.md` (workstreams, procedures, schedule, contingencies), `SIM\_MEASUREMENTS.md` (every parameter the simulator guesses and how to measure it), `INSTALL\_PLAN.md` (what to install and download, ordered by how badly it blocks us), `PLAN\_B.md` (the slow fallback). Sections 12–15 of this document summarise them.

Part I — Crash course

1 The competition in two pages

1.1 What the AI Grand Prix is

An autonomous drone-racing competition run by Anduril with the Drone Champions League (DCL); the drones come from Neros.^[ext] More than 3,000 teams entered. Human input during a timed run disqualifies it: the drone must fly itself.

Stage	When	What
VQ1 (virtual qualifier 1)	opened 31 May	DCL “FlightSim” simulator on Windows; short course; spec VADR-TS-001.
VQ2	29 Jun – 3 Aug	Harder course, position and attitude data blocked: camera (640×360) + IMU only. Top 10 teams advanced. ^[ext]
Physical Qualifier (PQ)	15–22 Sep 2026	Real drones in a hall at Anduril LC3 (OC-23, 3100 W Lake Center Dr, Santa Ana). Spec VADR-TS-005 (issue 00.02).
Finals (“AI Grand Prix Ohio”)	November 2026	Columbus, Ohio; live head-to-head; \$500K prize pool; top 10 at Ohio get at least \$5K. ^[ext] How many PQ teams advance is not published.

Table 1: Competition stages.

1.2 PQ rules that matter

- **Scoring** (spec §3.3): rank by the *maximum number of successfully passed gates*; tie-break is the *fastest elapsed time to the last validly crossed gate*. Qualification uses the best result from the **final two days** (Mon 21 and Tue 22 Sep).
- **A run** is two laps. Timing starts at the first timing line or gate trigger. Any pilot intervention makes the run invalid.
- **Practice**: several dedicated slots per team per day (15 min each in the earlier public spec TS-004^[ext]) plus time in a training cage. Manual piloting is allowed in practice.
- **Fleet**: each team gets 4 drones. Neros repairs them and does all charging; personal chargers are not allowed.
- **Flight controller access**: rates, PIDs, filters and telemetry may be changed through the Betaflight CLI; reflashing is not allowed.
- **Safety**: a pilot’s RC transmitter always overrides autonomy. Nobody stands near the flight path. The safety briefing on day 1 is mandatory.
- **Logistics**: 07:00–22:00 on site, out by 23:00, day badges, closed-toe shoes. Team 10 has its own password on the “AI Grand Prix” racing Wi-Fi. That password is in the welcome packet and must never go into the public repo.

1.3 The hardware you will fly

Airframe	Neros Archer B2, 8-inch propellers (4.1 pitch). All-up race weight measured on site, 17 Sep: 1745 g. ^[V] Motor and battery specs are given on site.
Flight controller	Betaflight. Another PQ team reads firmware 4.4.3 on an H743 board, MSP API 1.45. ^[ext]
FC ↔ computer	UART using MSP (MultiWii Serial Protocol), not MAVLink. Another team uses <code>/dev/ttyTHS1</code> at 115200 baud and sees about 100 MSP replies/s in total. ^[ext]
Computer	NVIDIA Jetson Orin NX 16 GB on a Seeed A603 carrier, JetPack 6.2 (Ubuntu 22.04, Python 3.10), 25 W mode, full root access. The organizers provide demo apps for camera bring-up and FC serial communication.
Camera	Arducam 12.3 MP HQ (Sony IMX477), rolling shutter, M12 lens, 1920×1080. Spec: 60 fps and 75° horizontal FOV, which is fx = fy = 417.0 px at 640×360. Another team calibrated <i>their</i> lens at $fx \approx 1271$ px (74.1°), which is close but is their lens, not ours. Tilt is adjustable <i>in flight</i> , exposure and gain are controllable, and no calibration matrices are provided — so we must calibrate our own.
IMU	From Betaflight, calibrated at power-on. Spec says 50 Hz; about 92 Hz is achievable over MSP. ^[ext]
Positioning	None: no motion capture, no GPS.

Table 2: PQ hardware (spec VADR-TS-005 §3.6–3.8 and FAQ).

1.4 Is the course the same as in our simulator? (FAQ)

Short answer: yes, it is a fixed, published course

The organizers publish one layout: the plan on page 6 of our spec, plus a gate-coordinate table (“Drone Racing Track — Gate Layout & Coordinate Reference for Contestants”). The course is built once in the hall and is the same for every team; it is *not* randomized between runs. Our Isaac track is a hand-digitized copy of that plan, slightly mis-scaled (Section 8.3).

What can differ in practice: (i) as-built placement a few tens of centimetres or a few degrees off the drawing; (ii) gates knocked and re-set during the week; (iii) exact heights, which the plan does not give (frames stand on the floor, so the opening centre is about 1.35 m; the double gate’s upper opening is about 4.05 m, confirmed on site on 15 Sep by another team^[ext]); (iv) the spec says map-distribution details “will be announced later”, so ask whether the layout can change before scoring. This is why the durability study treats “gate shifted a little” as *expected*, while “a different course” is only a stress probe.

2 Drone-racing autonomy in ten minutes

2.1 The loop

Every autonomous racer runs the same loop many times per second:

1. **Perceive.** Find the gates in the camera image. Our stack describes a gate by its **8 keypoints**: the 4 corners of the 2.7 m outer frame and the 4 corners of the 1.5 m opening.
2. **Estimate.** Know your own attitude (roll, pitch), rotation rates (gyro) and, roughly, your velocity. There is no position sensor, so the gate itself is the landmark.
3. **Decide.** A controller or learned **policy** maps what it sees to a command.
4. **Actuate.** Send the command to the flight controller (FC), which spins the motors.

2.2 Vocabulary

Term	Meaning
Roll / pitch / yaw	Rotation about the forward / right / down axes. Roll and pitch tilt the thrust to move sideways and forward; yaw turns the nose.
Body rates	How fast the drone rotates (rad/s) about each axis. The gyro measures them.

Term	Meaning
ACRO mode	Betaflight mode in which the sticks command <i>body rates</i> . Fastest and most agile. Our policy is trained for this.
ANGLE mode	Betaflight mode in which the sticks command <i>tilt angles</i> and the FC self-levels. Safer and slower. Plan B uses it.
Throttle / collective MSP	Total thrust. “Hover stick” is the throttle value that exactly holds altitude. MultiWii Serial Protocol. Betaflight’s command/telemetry protocol over a serial link.
MSP override	Betaflight feature: while an AUX switch is on, selected RC channels come from the companion computer over MSP instead of the pilot’s radio.
MAVLink	A different drone protocol (PX4/ArduPilot). The VQ simulator used it, which is why our client speaks it.
Jetson	The onboard NVIDIA computer that runs our code.
Keypoints	The 8 gate corners in pixel coordinates.
PnP	Perspective-n-Point: solve the camera’s pose relative to an object from its known 3-D corners and their pixel positions.
FOV / fx	Field of view, and focal length in pixels. Narrower FOV means larger fx and larger-looking gates.
Latency	Delay from photon hitting the sensor to motor response.
RL / PPO	Reinforcement learning: the policy learns by trial and error from a reward. PPO (Proximal Policy Optimization) is the algorithm we use.
Policy / checkpoint	The trained neural network, and the file that stores its weights (.pt).
Observation / action	What the policy sees / what it outputs.
Reward shaping	Extra reward terms that guide learning (e.g. “progress toward the gate”).
Privileged information	Information a simulator has that the real drone does not, e.g. exact gate corners.
Sim-to-real gap	Everything that differs between simulation and reality.
Domain randomization	Varying the simulator during training (mass, delays, noise) so the policy tolerates reality.
Isaac Sim / Isaac Lab	NVIDIA’s GPU physics simulator, and its robot-learning framework (thousands of drones in parallel).
skrl	The RL library used to run PPO.
Keeper	The team’s word for the saved model currently worth keeping; here, Gene’s model <code>pq_speed_best.pt</code> .
Episode / attempt	One simulated flight, from its start until a crash or the time limit.
Plant	The simulator’s model of how the drone physically responds to commands.
Headless	Running the simulator without its 3-D window.
Fine-tune / warm start	Continue training from an existing saved model instead of from scratch.
IMU	Inertial measurement unit: the gyroscope (rotation speed) and accelerometer on the flight controller.
Betaflight	The flight-controller firmware on the drone. It stabilizes the drone and turns stick commands into motor speeds.
UART / serial	The cable link between the Jetson computer and the flight controller.
AUX channel	An extra radio channel driven by a switch on the pilot’s transmitter (for example ARM on AUX1).
Bitmask	A number whose binary digits switch individual options on or off (the override mask 15 = binary 1111 = roll, pitch, throttle and yaw).
Failsafe	What the flight controller does when it loses its command link; here DROP cuts the motors.
PID	A standard feedback controller that corrects an error using its size, its accumulated history and its rate of change.
Barometer	An air-pressure sensor that gives altitude.
Dead reckoning	Estimating position by integrating motion estimates, with no external reference; errors grow over time.
Gate normal / axis	The direction a drone must fly through a gate, perpendicular to the gate frame.
p95	The 95th percentile: 95 % of values are at or below it.
Intrinsics / calibration	A camera’s focal length, image centre and lens distortion, measured with a printed pattern (for example a ChArUco board).

Term	Meaning
YOLO / HSV detector	Two ways to find gates in an image: a neural-network detector (YOLO) or a colour threshold (HSV).
TensorRT / GStreamer	NVIDIA's tool for running neural networks fast on the Jetson; a video pipeline library for reading the camera.

3 How our system works

3.1 Big picture

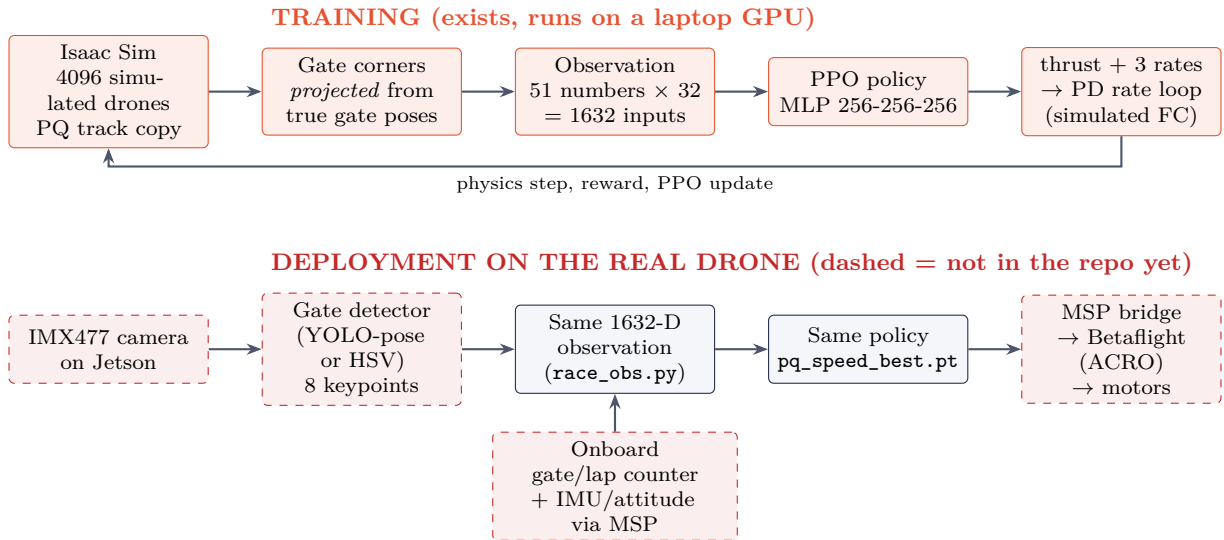


Figure 1: Training vs deployment. The policy and observation format are shared; the dashed blocks are the missing deployment layer.

3.2 Reinforcement learning in plain terms

How Gene’s model was made

- **The pilot is a neural network**, called the *policy*. Many times per second it receives a list of numbers describing what the drone currently senses (the *observation*) and outputs four numbers: throttle and three rotation speeds (the *action*).
- **Nobody tells it how to fly**. It starts out flying randomly inside a simulator. After every step the simulator hands out a score, the *reward*: points for moving toward the next gate and for passing through it, and a penalty for crashing.
- **The training algorithm, PPO** (Proximal Policy Optimization), slowly adjusts the network’s internal numbers so that actions which earned more reward become more likely.
- **Scale**: Isaac Sim runs thousands of simulated drones in parallel on the graphics card (4096 in Gene’s runs).
 - A *timestep* is one decision made by all of those drones at once.
 - *Samples* are timesteps multiplied by the number of drones.
 - One of Gene’s typical training runs is 50,000 timesteps, about 205 million samples, about 1.4 hours.
- **Checkpoints**: the saved network is a *checkpoint* file (.pt). Training runs can start from an earlier checkpoint instead of from scratch (a *warm start* or *fine-tune*).
- **The key limitation**: the network only learns to handle what the simulator showed it. If the simulator’s course, camera, delays or thrust differ from reality and were never varied during training, the network has no reason to cope with the difference. Section 9 measures exactly this.

3.3 What the policy sees and does

Slots	Content	Notes
0–15	where the 8 gate corners appear in the camera image (horizontal and vertical pixel position, divided by the image size 640×360)	–1 if a corner is not visible; order: outer frame top-left, top-right, bottom-right, bottom-left, then the same for the inner opening
16–23	visibility flags	corner in front of camera and within frame $\pm 15\%$
24–25	roll, pitch (rad)	gravity-referenced
26–28	rotation speeds from the gyroscope, limited to ± 8 rad/s	
29–31	“commanded” body velocity	integrated from thrust, attitude and a drag model; never from the accelerometer
32–49	which gate is next: 18 numbers, all 0 except a single 1 at that gate’s position (11 of the 18 slots are used on this course)	the policy always knows which gate it is flying to
50	progress through the course: gate number divided by 17 (the list has 18 slots, numbered 0–17)	

Table 4: One observation frame (51 numbers). The policy receives the last 32 frames (0.53 s at 60 Hz): 1632 inputs. Keypoints refresh at 30 Hz; everything else at 60 Hz.

How the model is structured

- **Network:** a plain feed-forward net, $1632 \rightarrow 256 \rightarrow 256 \rightarrow 256$ (ELU), with a 4-number action head and a 1-number value head (551k parameters). Inputs are normalised by stored running means and variances.
- **Output:** $[-1, 1]^4$, decoded to
 - a throttle stick: hover at 0.255, range 0.05–0.90, mapped linearly to newtons with a 0.6076 kg mass (maximum thrust-to-weight ≈ 3.5);
 - three body rates of up to ± 3.2 rad/s.
- **No map input**, but the “which gate is next” numbers tell it *where on the course* it is. That is how it learns this specific course’s turns. It also means a different course, or a wrong gate count on the real drone, will confuse it.

3.4 How it was trained

- **Timing:** 120 Hz physics, 60 Hz policy, 40 s episodes, a random start gate each episode, tiny random pushes (± 0.1 N).
- **Plant:** rigid body with collective thrust plus a PD body-rate loop ($k_p=0.08$, $k_d=0.003$, moment limits 0.30/0.30/0.20 N·m). No motor model, no drag, no latency, no sensor noise.
- **Episode ends on:** collision (including the ground), crossing a gate plane outside the ± 0.75 m opening, more than 20 m from the target gate, or the 40 s time-out.
- **PPO settings (technical; safe to skip):** 4096 envs, 24-step rollouts, learning rate 10^{-4} (KL-adaptive), $\gamma=0.99$, $\lambda=0.95$, 50k timesteps per run (≈ 1.4 h on Gene’s RTX 4070 laptop with 4096 drones).

Reward term	Weight	Idea
gate passed	+600	the real objective
progress toward the next gate / toward a point 3 m straight in front of it	+20 / +25	continuous guidance toward the gate
speed along the gate normal at the crossing	+3.4	go fast through gates
camera aligned / centred at crossing	+4 / +4	pass cleanly
low-gate dive, proximity, heading, look-at, visible	small	shaping
time penalty, angular-rate penalty	-0.01 , -10^{-4}	
termination (crash, miss, flyaway)	-100	

Table 5: Reward terms: the points the simulator awards during training (current code, identical to the settings Gene’s model was trained with).

4 Repo tour

Path (branch isaacsim)	What it is
ISAACSIM.md	Start here for the PQ stack: Windows GUI play, WSL headless training, installs
isaac_drone_racer/	The Isaac Lab project (fork of kousheekc/isaac_drone_racer, copied in without its git history)
tasks/drone_racer/drone_racer_env_cfg.py	Scene, track , observation, rewards, terminations, task settings
tasks/drone_racer/mdp/	Actions (plant), commands (gate targeting, pass detection), observations, rewards, events
utils/aigp_obs.py	Observation contract: camera model (FX=320, 20° tilt), keypoint projection, velocity integrator
dynamics/rate_control.py	Action decode, stick→newtons, PD rate moments
scripts/rl/train.py, play.py	Training and GUI playback
models/pq_speed_best.pt	Gene's best model (plus earlier saved models)
logs/skrl/drone_racer/	All 32 training runs, 30 Aug – 6 Sep (TensorBoard + checkpoints, 1.5 GB)
PHYSICAL_TUNING.md	On-site procedure to measure the real drone and copy constants into Isaac
race_obs.py, isaac_planner.py	The same observation on the flight-client side; runs an Isaac policy inside the client
main.py, setup.py, controller.py,	Flight client written for the VQ simulator (MAVLink/UDP)
mavlink_rx.py, vision_rx.py	
tools/pad_constants.py, tools/run_isaac.py	On-site measurement tool, Isaac-policy runner (both MAVLink)
vision/	Gate detectors: YOLO-pose, deterministic HSV orange-opening detector, PnP
reference/	VQ and PQ specs, papers

Table 6: Where things live. Behaviour-cloning / HG-Dagger code and the VQ-simulator models are retired and not covered.

4.1 Running things on the team laptop (WSL)

```
# Isaac environment (Isaac Sim 4.5, Isaac Lab 2.1, skrl 1.4.2, Python 3.10)
source ~/aigp/isaac_env.sh
# train or evaluate from a scratch copy, never from the repo (training writes logs/ into cwd)
cd ~/aigp/idr_exp # copy with the gate-counting fix applied
python -u scripts/rl/train.py --task Isaac-Drone-Racer-v0 --headless --num_envs 2048 --max_iterations 30
# headless evaluation / stress tests (scratch tools, outside the repo)
PYTHONPATH=$PWD:$PYTHONPATH python ~/aigp/tools/eval_policy.py --headless \
  --task Isaac-Drone-Racer-Play-v0 --num_envs 1 --steps 4800 \
  --checkpoint models/pq_speed_best.pt --out ~/aigp/eval/play.json
PYTHONPATH=$PWD:$PYTHONPATH python ~/aigp/tools/stress_eval.py --headless --num_envs 512 \
  --checkpoint models/pq_speed_best.pt --scenario '{"track":"official","cam_fx":423.6}' --out out.json
```

Isaac Sim often hangs on exit after writing results; kill it by PID. The 3-D viewer does not render under WSL (no Vulkan device); watching needs the Windows install described in ISAACSIM.md.

5 The physical course

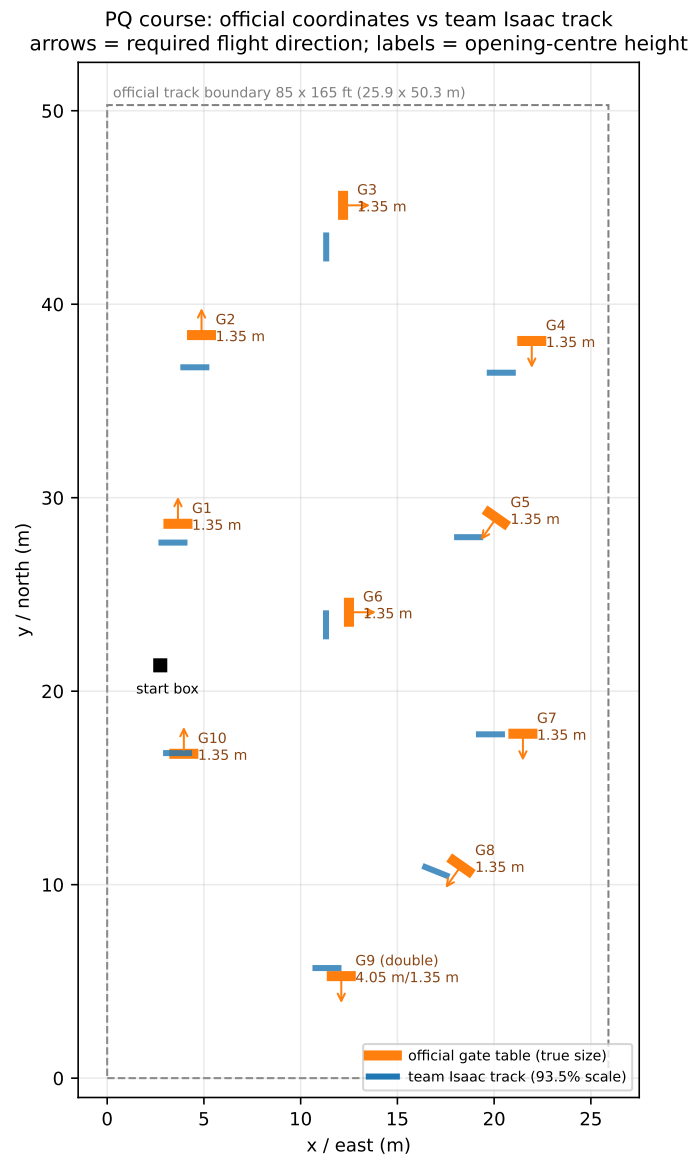


Figure 2: Official gate coordinates (orange, true size) against the team's Isaac track (blue). The Isaac copy is a roughly 93%-scale version of the official layout (Section 8.3).

Leg	Distance	Arrival angle off gate axis	Gate number in Gene's simulator	Note
start box → G1	7.4 m	7°	6	takeoff from the floor (never trained)
G1 → G2	8.3 m	8°	7	
G2 → G3	9.0 m	35°	8	
G3 → G4	10.8 m	50°	9	
G4 → G5	7.9 m	22°	10	G5 faces 215°
G5 → G6	7.7 m	152°	0	split-S (the drone passes beside G6, turns around, and comes back through it): pass north of G6, turn back east
G6 → G7	9.8 m	50°	1	
G7 → G8	6.2 m	4°	2	G8 faces 215°
G8 → G9 top	7.0 m	50°	3	climb to 4.05 m, fly south
G9 top → G9 bottom	turnaround	180°	4	descend to 1.35 m, fly north
G9 bottom → G10	13.6 m	32°	5	longest leg
G10 → G1	10.4 m	2°	6	lap 2

Table 7: Leg geometry from the official table (straight-line, plan view). Headings: the official table gives compass bearings (clockwise from north, so 215° points south-west); elsewhere this document uses the mathematical convention (counter-clockwise from east, so the same heading is -125°). A complete run is two laps; one lap is 11 crossings. Another team counts 23 crossings, finishing on gate 1.^[ext]

Part II — Engineering journal

6 Project history

Dates	Branch	Approach	Outcome
3 Jun	main, modified-starter	Organizer example client; HSV colour gate detection; attitude + thrust commands	controller working, untuned
6–16 Jun	preplanning	reactive gate chasing, then fixed waypoints	gate 1 passed
17–20 Jun	manual-control, spline-path	fly by keyboard to record the course, replay as a spline (VQ1 gave position data)	“sub 14 seconds” (commit message only)
28 Jun–18 Jul	Qualifier2*	VQ2 blocked state data: IMU + vision rebuild	gap analysis: “vision not connected to flight”; abandoned
23–25 Jul	Q2_new	DreamerV3 (model-based RL) against the real-time simulator	168k steps, 0 gates; deleted
26–27 Jul	Q2_new, Q2_CV, Q2_pnp	OpenCV and YOLO corners, PnP + IMU	passed gates 1–2
28 Jul–1 Aug	Q2_kalman	dual-gate Kalman filter, tuning harness	human best 35.96 s
10–30 Aug	classical-vision-racing	classical Li & de Croon stack; behaviour cloning / HG-Dagger (retired as weak)	human best 14.04 s; no autonomous lap recorded
30 Aug–16 Sep	isaacsim	Isaac Lab PPO on the PQ course	best model pq_speed_best (5 Sep); copied into the AI_GP repo on 16 Sep

Table 8: Eras of the project. The `isaacsim` branch contains every useful earlier branch.^[V]

Where the Isaac work was done. All 32 runs record their directory as `/mnt/d/Code/Competitions/isaac_drone_racer/...`^[V], a separate folder on Gene’s machine. It was copied into AI_GP on 16 Sep without its own git history. Gene reports everything relevant is pushed. Trained models for the retired VQ path are excluded from git on purpose and are not needed.

7 Isaac training history and Gene’s best model

Run	Change	Start from	Best (gates/ep, reward)
30 Aug – 2 Sep	7- and 17-gate VQ-style tracks, observation and action redesigns, several NaN/abort runs	scratch	1.5–4.5 gates
2–3 Sep	look/visibility reward experiments (policy learned to hover and stare)	scratch	≈0 gates
3–4 Sep	zig-zag and “passage” shaping (pass_main, vel_align, passage_shape)	chain	1.8–5.9
4 Sep 14:49	first PQ track (different gate order)	passage_shape	0.5 (transfer failed)
4 Sep 17:03	PQ track, current order, gate-passed 600	scratch	25.3, 356 (pq_scratch)
4 Sep 20:38	+ run-in progress, velocity alignment	pq_scratch	30.8, 427 (pq_vel)
4 Sep 23:48	run-in progress 25	pq_vel	28.9, 400 (pq_hook)
5 Sep 01:08	speed-through-gate 3.4	pq_hook	33.2, 463 (pq_speed_best)
6 Sep 01:35	identical settings, continued	pq_speed_best	0.0 — collapsed (pq_hook2)

Table 9: Training lineage (from checkpoint hashes, run configs and TensorBoard).^[V] “Gates/ep” is per 40 s training episode with exploration noise.

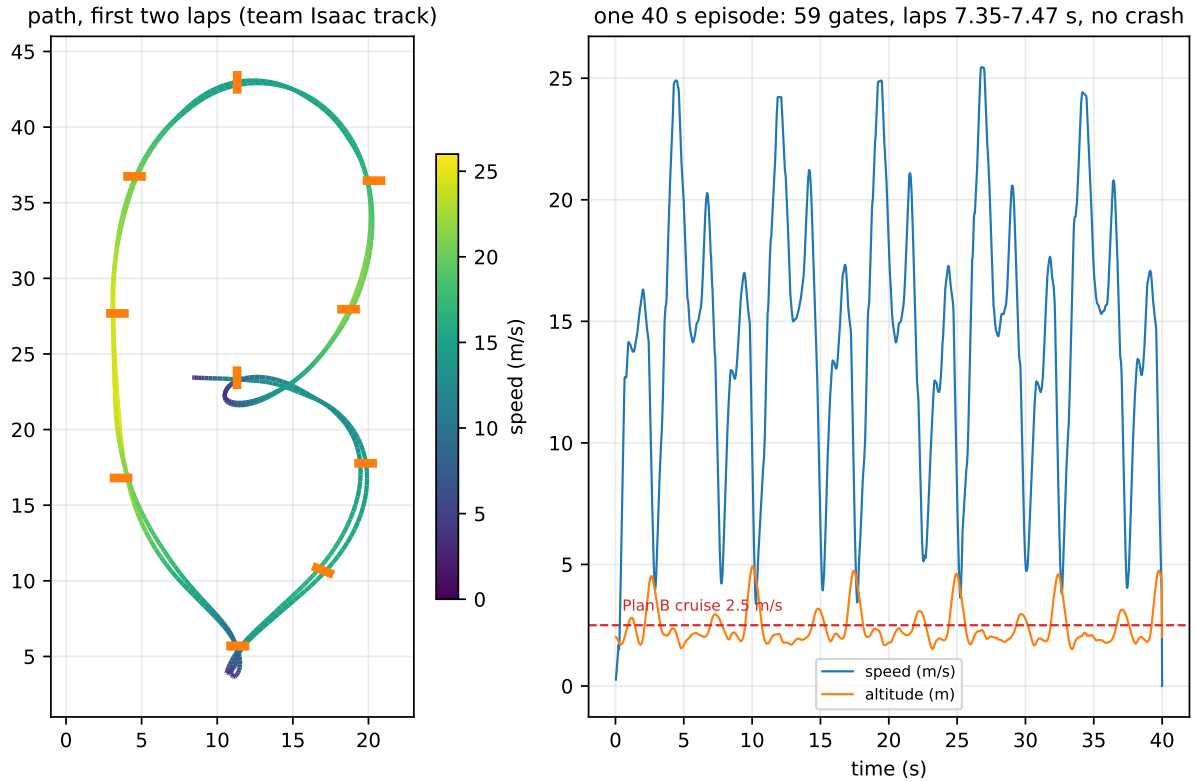


Figure 3: Gene’s model flown on 16 Sep without a display, in Isaac’s playback mode (fixed start position, no disturbances, and the network’s single best action at every step rather than random exploration).^[V] 59 gates in 40s, laps 7.35–7.47 s, mean 14.4 m/s, peak 25.4 m/s, altitude 1.5–4.9 m. This reproduces the 7–8 s lap time Gene reported from his own runs. The red dashed line marks Plan B’s gate speed for comparison.

8 Audit findings

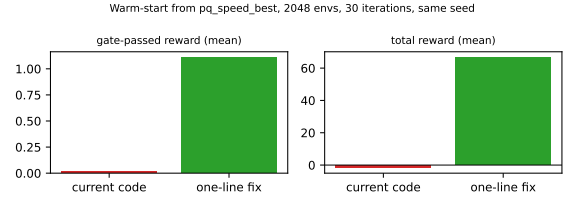
Item	Gene’s simulator (what the model was trained on)	Real physical qualifier	Fixed by the corrections patch?
Camera view width	90° (focal length 320 px)	74°–75° (about 424 px)	yes
Camera tilt	exactly 20° up	adjustable; unknown until measured	no; measure on site
Course size	about 93 % of true size	official coordinates	yes
Gate heights	opening centres at 2.0 m (double gate 4.7 m)	about 1.35 m (double gate about 4.05 m)	yes
Start	in the air, in front of gate 6	on the floor, 7.4 m before gate 1	playback start only; takeoff never trained
Delay from camera to motors	none	roughly 50–130 ms	no; needs training with delay
Stick response	straight-line mapping	Betaflight curves (nonlinear)	no; change FC settings or convert in code
Drag, sensor noise, bumps	none (tiny random pushes only)	present	no; needs training with them
Gate corners	computed exactly from the true gate position	must be found by a camera detector (none exists yet for real gates)	no
Which gate is next	provided by the simulator	must be counted onboard (code does not exist yet)	no
Communication	MAVLink over UDP	MSP over a serial cable to Betaflight	no; new code needed
Counting gate passes during training	broken on any reset (bug)	n/a	yes

Table 10: Where Gene’s simulator differs from the real event. The durability tests in Section 9 measure how much each difference matters.

8.1 Gate-counting bug in the current Isaac code

tasks/drone_racer/mdp/commands.py:234: whenever *any* environment resets, `prev_robot_pos_w` is overwritten for *all* environments. Isaac Lab 2.1 then runs the gate-crossing check in the same step (reset first, then `command_manager.compute`), so previous equals current and no crossing is registered anywhere.^[V] With thousands of parallel drones, resets happen almost every step. That explains why the 6 Sep continuation of Gene’s model, with byte-identical settings, logged zero passes *and* exactly zero misses.

```
# candidate fix (applied only to scratch copies)
prev = self.prev_robot_pos_w.clone()
prev[env_ids] = self.robot.data.root_pos_w[env_ids]
self.prev_robot_pos_w = prev
```



Left: points earned for passing gates; right: total points; both averaged per training log window, before and after the fix.

Verdict

Confirmed by A/B test.^[V] Starting from Gene’s model with identical random seeds, the current code logs 0.014 mean gate reward and 0 misses; with the fix it logs 1.11 and 0.16. Total reward is -1.6 vs $+66.7$. Any on-site retraining (which `PHYSICAL_TUNING.md` plans) collapses until the fix is in. Playback with one drone is mostly unaffected. Clone before writing, because line 327 stores Isaac’s buffer without copying it.

8.2 Camera field of view: who is right?

- **Code:** `utils/aigp_obs.py` and `camera_model.py` use a 640×360 image with a focal length of 320 pixels (written `fx = fy = 320`; the focal length in pixels sets how wide the view is) and a 20° upward camera tilt. That is a 90.0° horizontal FOV, taken from the *VQ* simulator spec.^[V]
- **PQ spec** (VADR-TS-005 §3.8): 75° horizontal FOV at 1920×1080 .
- **Measurement**^[ext]: AI of Sauron calibrated the same camera model with a ChArUco board, on their own lens: `fx = 1270.7 px`, `fy = 1267 px`, principal point (976, 497), five distortion coefficients, and 0.28 px reprojection error. That is $74.1^\circ \times 46.2^\circ$.

Same gate and pose, two camera models (640x360 image, 20 deg tilt, gate 0.8 m right at eye level)
solid = 2.7 m frame, dashed = 1.5 m opening; lines leaving the box are out of view

code / VQ camera: `fx=320` (90 deg HFOV)

PQ camera (spec 75 deg; measured `fx~424` at 640 px)

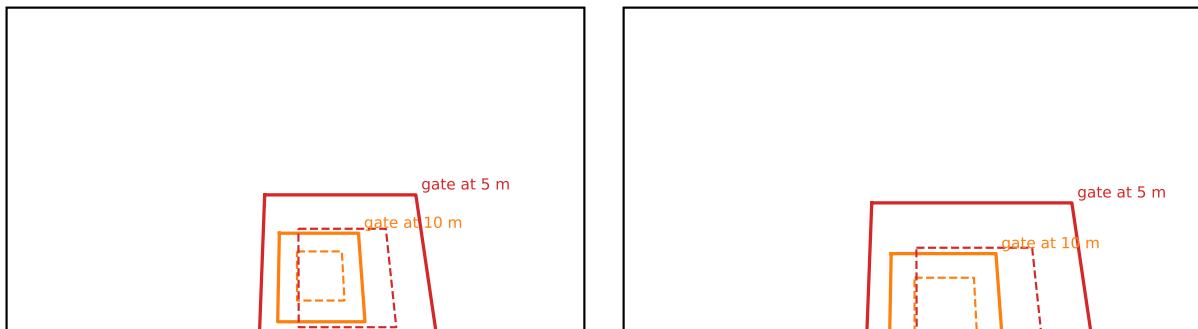


Figure 4: The same gate rendered with the code’s camera and with the PQ camera. Gates appear about 32% larger and farther off-centre on the real camera, which is exactly the input the policy normalises.

Verdict

Our code is wrong for the PQ; the spec and the measurement agree. The specification is the authority and it says 75° horizontal, which at 640 px width is a focal length of **417.0 px**, not 320. (The corrections patch of 16 Sep used 423.6 px, i.e. 74.1° , taken from another team’s calibration of their own lens; **use 417.0** until we calibrate ours.) The real principal point also sits about 14 px above centre, and the lens distortion is significant. A 74° image cannot be remapped to 90° , so the policy must either be retrained with the real intrinsics or shown to tolerate the change (Section 9). Camera tilt on our drones is still unmeasured. Calibrate on day 1; it takes about 15 minutes.

8.3 Track size and gate heights: who is right?

The Isaac track comment says it was digitized by assuming the pylons are 15 m apart. We fitted the 11 Isaac gate centres to the organizer’s numeric gate table with a similarity transform (scale, rotation, translation):

- best fit: **scale 0.9348**, rotation 0.03° , all gates within 0.42 m;
- forcing scale = 1 leaves errors up to 1.47 m.^[V]

Headings agree except gate 5 (Isaac -90° vs official -125° , off by 35°) and gate 8 (off by 13°). The official boundary, 85×165 ft = 25.9×50.3 m, also matches the “ 25.9×50.3 m” label on the spec plan.

Verdict

The Isaac track is 6.5 % too small, and two gate headings are off. The organizer table is numeric; the Isaac copy was measured off a picture with an assumed ruler. **Heights:** Isaac places opening centres at 2.0 m (double gate 4.7 m). Floor-standing 2.7 m frames put them at 1.35 m (double gate 4.05 m), as confirmed on site by another team.^[ext] **Start:** Gene’s simulator numbers the gates starting from the centre gate (official gate 6 is its gate 0), and its playback mode starts in the air in front of that gate. The real race starts on the floor, 7.4 m before official gate 1, which is gate 6 in Gene’s numbering. Confirm with a tape measure on day 1: one gate-to-gate distance settles the scale.

Unresolved as of 17 September: the hall footprint

The specification (§3.4) states the track footprint is **60 m $\times$ 21 m**. The plan drawing, measured with its own printed ruler, gives **25.9 m $\times$ 50.25 m**. Those do not reconcile by any single scale factor — the spec’s track is *longer* and *narrower*. Most likely the spec quotes the usable hall volume and the drawing quotes the gate envelope, but that is a guess. **The on-site survey settles it, and until it does, no document should be trusted for absolute course size.**

Clarification on drone mass: the *simulated* drone’s mass does not differ from the mass its own thrust model assumes — both are 0.6076 kg.^[V] There is no internal mismatch. The real drone weighs 1745 g, which is a different problem entirely and is explained in the status section at the front of this document.

8.4 Flight-controller configuration

From another team’s full dump all of the same drone model^[ext]:

- **Firmware and modes:** Betaflight 4.4.3 (H743). ARM is on AUX1 (pilot’s CRSF radio) and **MSP OVERRIDE** is on AUX5. No ANGLE/HORIZON mode is assigned, so it flies ACRO.
- **Override mask:** `mcp_override_channels_mask` = 11, i.e. roll, pitch and yaw only. **Throttle is not overridden**; autonomy needs 15.
- **Rates:** Betaflight rates with `rc_rate` 0.55 and `super_rate` 0.75, about 440°s^{-1} at full stick and **nonlinear**. The policy assumes linear $\pm 183^\circ \text{s}^{-1}$.
- **Throttle curve:** `thr_mid` 54, `thr_expo` 68, **nonlinear**. The plant assumes linear.
- **Failsafe:** `failsafe_procedure` = DROP.
- **Units:** MSP raw gyro is in counts (16.4 per $^\circ \text{s}^{-1}$), not $^\circ \text{s}^{-1}$.

Safety fact from the Betaflight 4.4.3 source

In `src/main/rx/msp_override.c`, while the override switch is on, the FC uses the **last** `MSP_SET_RAW_RC` values it received, **with no timeout**.^[ext] If the Jetson process hangs mid-flight, the drone keeps flying its last command until the pilot flips the switch off or disarms. Stream a safe neutral frame before enabling override, run a separate stall watchdog, and brief the pilot.

8.5 The missing deployment layer

This is the critical path, confirmed 17 September

A repo-wide search finds no occurrence of MSP anywhere in the flight client.^[V] `controller.py` sends MAVLink `SET_ATTITUDE_TARGET` to a `sim_conn` — the virtual qualifier’s simulator interface. Betaflight does not accept MAVLink attitude setpoints, and the PQ specification confirms the real interface is **MSP over UART**. **Nothing we have can currently send a single command to the real drone**. Every other activity that involves flying is downstream of fixing this.

A repo-wide search also finds no occurrence of serial, Betaflight, Jetson, TensorRT, GStreamer/Argus or `CLOCK_MONOTONIC` in the client.^[V] To fly anything on the real drone we need:

1. an MSP bridge (commands out; attitude, raw IMU, status and battery in);
2. Jetson camera capture with capture timestamps;
3. a gate detector that works on *real* gate images (the existing detectors were tuned on simulator images);
4. an onboard gate/lap counter, because the policy’s “next gate number” input came from the simulator and both learned planners raise an error without it;^[V]
5. real-drone safety: crash detection, arming, finish and landing. The existing logic is simulator-based and auto re-arms after a crash.^[V]

Both Plan A and Plan B need items 1, 2, 3 and 5.

8.6 Smaller items

- **Parity tests:** the two `AI_GP↔Isaac` observation parity tests silently skip after vendoring (path moved). With the path fixed, all 29 pass.^[V]
- **Bridge defaults:** the bridge’s default weights path points to a VQ-era file; use `pq_speed_best.pt` explicitly. It also enables a “visual snap” history rewrite that training never used.^[V]
- **Stale scripts:** `scripts/r1/probe_race.py` references a removed reward term and will crash.^[I]

9 Durability study (16 Sep)

9.1 Method

- **Setup:** each test (“scenario”) runs Gene’s model, always taking its single best action, on 512 simulated drones at once for 40 s (2400 control steps); tests run later in the day used 20 s.
 - Each simulated drone flies until it crashes, is immediately reset, and tries again. One start-to-crash flight is an *attempt*.
 - Unless a row says “race start”, each attempt begins just past a randomly chosen gate, as in training. “Race start” means every attempt begins in front of official gate 1, like the real race.
 - Every test uses a copy of Gene’s code with the gate-counting bug fixed. The changes being tested are applied at runtime; none of the team’s files were modified.

How to read the numbers in the tables below

- **Gates per attempt:** how many gates a drone gets through, on average, before it crashes. Higher is better. In the 20–40s tests a good model is capped by the test length (about 20–28).
- **Crashes per 100 gates:** total crashes divided by total gates passed, times 100. A crash is hitting anything (including the floor), crossing a gate’s plane outside its opening, or flying more than 20 m away from the next gate. Crashes in the first 1.5 s after a random start are ignored. Lower is better.
 - This is *not* a percentage, so it can exceed 100. A value of 110 means the drone usually crashes before it passes even one gate.
- **Chance of two clean laps** (written $P(2 \text{ laps})$): the probability of passing the 22 gate crossings of two laps (11 per lap, the double gate counting twice) without a single crash, estimated as $\max(0, 1 - \text{crashes per gate})^{22}$. It is 0 % whenever crashes outnumber gates.
- **Test length:** 40 s for the first tests, 20 s for tests run later in the day, 25 s for the retraining comparison. Crashes per 100 gates can be compared across lengths; gates per attempt only within the same length.
- **Test labels:** codes such as “A6” or “S1” are just short names for individual tests. Their meaning is written out in each row.
- **Realism levels** (the “Band” column):
 - **Reference:** Gene’s own simulated course, as trained.
 - **Expected:** things that will almost certainly be true at the real event:
 - * the true course size and gate heights;
 - * the real camera;
 - * gates placed a little off the map;
 - * imperfect gate detection;
 - * about 50 ms of delay between seeing and acting;
 - * a hover thrust that is slightly wrong;
 - * a heavier, slower-responding 8-inch drone;
 - * air drag.

There is no wind, because the event is indoors.
 - **Rougher:** plausible but pessimistic versions of the same things, plus small bumps (propeller wash, a light touch).
 - **Stress only:** things not expected at the event: wind, hard bumps, 200 ms delay, resized gates, and completely different courses. The last group tests whether the model memorized the course rather than learning to fly.
 - **Combined:** rows marked “combined” apply all the disturbances of their level at once.

9.2 Results

Realism level	Test	Gates per attempt	Crashes / 100 gates	$P(2 \text{ laps, no crash})$	Median lap (s)	Speed at gate (m/s)	Lateral offset p95 (m)
Reference	R0 baseline (Gene’s course, small random pushes as in training)	21.9	2.6	56 %	7.43	15.98	0.466
Reference	R1 ideal (Gene’s course, no pushes)	27.8	1.7	69 %	7.43	16.03	0.465
Expected	A1 official course: true size and floor-standing gates	7.9	9.9	10 %	8.07	15.73	0.494
Expected	A1b official size, Gene’s gate heights	8.0	9.8	10 %	8.07	15.71	0.492
Expected	A1c Gene’s size, floor-standing gate heights	26.5	1.8	67 %	7.43	16.03	0.465
Expected	A2 real camera: 74° field of view	1.1	42.4	0 %	–	13.96	0.558
Expected	A3 camera tilt 15°(-5)	1.9	32.1	0 %	7.85	13.52	0.582
Expected	A4 gate placement $\pm 0.25 \text{ m}$, $\pm 5^\circ$	9.5	6.3	24 %	7.52	15.88	0.49
Expected	A5 gate-detection noise 2 px, 5% missed frames	9.8	5.4	30 %	7.52	15.73	0.48

Realism level	Test	Gates per attempt	Crashes / 100 gates	$P(2 \text{ laps, no crash})$	Median lap (s)	Speed at gate (m/s)	Lateral offset p95 (m)
Expected	A6 latency 50 ms (vision 33 + command 17)	0.6	55.2	0 %	–	14.53	0.592
Expected	A7 thrust calibration -10%	1.0	57.7	0 %	–	14.03	0.567
Expected	A8 slower airframe (response lag 40 ms)	6.9	9.7	11 %	7.52	16.44	0.507
Expected	A10 8-inch drone clearance (must pass within 1.1 m box)	14.0	2.9	52 %	7.43	16.06	0.467
Rougher	B1 gate placement $\pm 0.5 \text{ m}$, $\pm 10^\circ$, $\pm 0.2 \text{ m z}$	5.1	14.5	3 %	7.65	15.73	0.514
Rougher	B2 gate-detection noise 5 px, 15% missed, 5% corners	2.0	26.9	0 %	7.96	14.91	0.546
Rougher	B3 latency 100 ms	0.1	94.7	0 %	–	9.52	0.446
Rougher	B6 gate detection only 15 times per second	5.1	13.0	5 %	7.71	15.95	0.522
Rougher	B7 small bumps (every 3 s)	14.8	2.8	53 %	7.43	16.06	0.472
Stress only	C4 gates 20% smaller	5.9	11.3	7 %	7.6	15.72	0.413
Stress only	C5 mirrored course	0.7	73.4	0 %	–	15.29	0.53
Stress only	C7 random course	0.7	46.9	0 %	–	13.02	0.564

Table 11: Durability results for Gene’s model (`pq_speed_best`). 512 simulated drones per row; the model always takes its single best action; the gate-counting bug is fixed. Tests R0–A3 ran for 40s and the rest for 20s, which also caps their gates per attempt. How to read the columns: see the box above. Speed at gate: average speed while passing through a gate. Lateral offset p95: in 95 % of passes, the drone’s centre was at most this far sideways from the centre of the 1.5 m opening (0.75 m would be touching the edge).

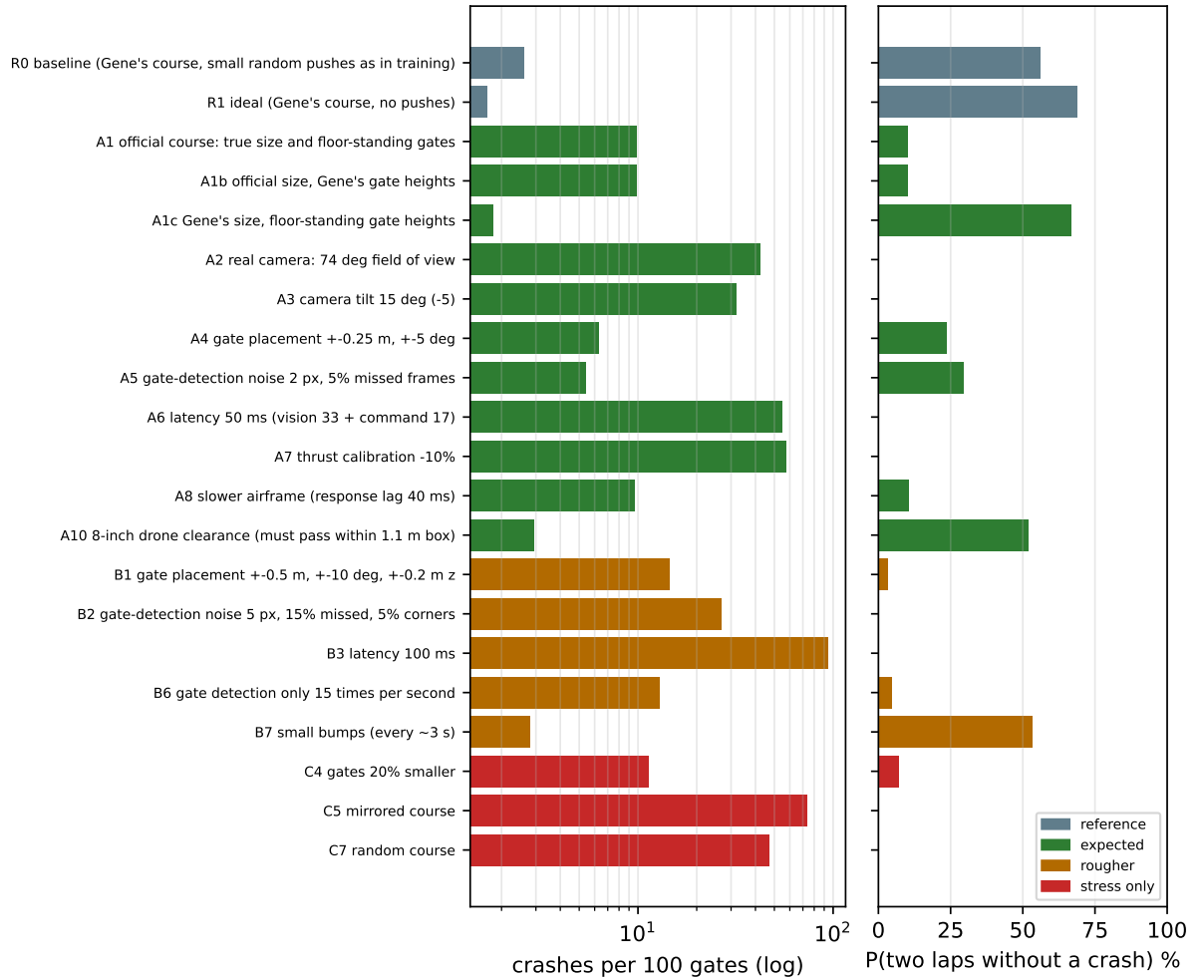


Figure 5: Crash rate per 100 gates (log scale) and the implied chance of finishing two laps, by scenario and realism band.

One change at a time to Gene’s model on Gene’s own course (unchanged: 2 crashes per 100 gates)	Crashes / 100 gates	$P(2 \text{ laps})$
Thrust calibration -10%	58	0 %
Latency 50 ms	55	0 %
Real camera (fx 424)	42	0 %
Camera tilt -5°	32	0 %
Spec drawing exactly (Gene’s heights)	16	2 %
Scale only (Gene’s map $\times 1.075$)	16	2 %
Gates 20 % smaller	11	7 %
True course size	10	10 %
Slower airframe (40 ms)	10	11 %
Gate placement $\pm 0.25 \text{ m}$	6	24 %
Detector noise 2 px	5	30 %
8-inch clearance	3	52 %
Correct gate heights only	2	67 %
Latency 100 ms (rougher)	95	0 %
Vision 15 Hz (rougher)	13	5 %
Detector noise 5 px, 15 % missed (rougher)	27	0 %
Small bumps (rougher)	3	53 %
Gate placement $\pm 0.5 \text{ m}$ (rougher)	15	3 %
Mirror-image course (stress only)	73	0 %
Randomly generated course (stress only)	47	0 %

Table 12: The sensitivity of Gene’s model to single realistic factors, ranked.

What the durability tests say about Gene’s model

- **Most damaging factors:** latency and thrust calibration. Just **50 ms** of total delay (55 crashes per 100 gates), or a hover thrust **10 % weaker** than assumed (58), is enough to make two laps essentially impossible. The real Jetson→MSP→Betaflight path will have at least that much delay (another team budgets about 130 ms), and hover thrust will only be known after measurement.
- **Camera model:** the real field of view (42) and a 5° tilt error (32) are next.
- **Course geometry:** scaling Gene’s own map up by 7.5 %, with nothing else changed, raises crashes from 2 to 16 per 100 gates. About 60 % of those crashes are on the climb to the *upper opening of the double gate*: the policy learned a manoeuvre timed to the exact distances it trained on. Heights alone do not matter (2).
- **Mild factors:** gate placement ($\pm 0.25 \text{ m}$), detector noise (2 px) and the larger 8-inch airframe are comparatively mild, at 6, 5 and 3 crashes per 100 gates.
- **It memorized this course.** On a mirrored or random course it crashes almost immediately (73 and 47). That is expected: its input includes “which gate is next”, so it learns the course, not a general skill. This is fine for a fixed course only if every other detail of the course and the drone matches training.
- **Is the 7–8 s lap time itself the problem?** Not directly; the speed is a symptom. The real problem is a policy optimized for an idealized plant with zero latency, exact thrust, and a camera and course that differ from reality. A retrain needs randomization of exactly the top factors above: latency (0–150 ms), thrust scale ($\pm 15\%$), camera tilt ($\pm 5^\circ$), rate lag. It also needs the corrected camera and course (Section 9.3).

9.3 Retraining on the corrected simulator (16 Sep)

Setup. As a feasibility test, Gene’s model was trained further (fine-tuned) for one hour on bojro’s laptop in a corrected copy of the simulator (the `pq_corrections.patch` changes):

- **Corrections:** official track and heights, the 74° camera, and the gate-counting fix.
- **Randomized “expected” disturbances**, so the model sees them during training:
 - 33 ms vision and 17 ms command latency;
 - 2 px keypoint noise with 5 % missed frames;
 - attitude and gyro noise;

- a 30ms rate lag and drag;
- a per-drone thrust scale of $\pm 10\%$;
- per-drone gate shifts of ± 0.25 m and $\pm 5^\circ$.
- **Size:** 2048 drones, 32.4k PPO timesteps (about 66M samples), roughly $\frac{1}{3}$ of one of Gene’s standard 50k-step, 4096-drone fine-tunes.

continued training of Gene’s model on the corrected simulator (official course, 74 deg camera, expected disturbances randomized), 2048 simulated

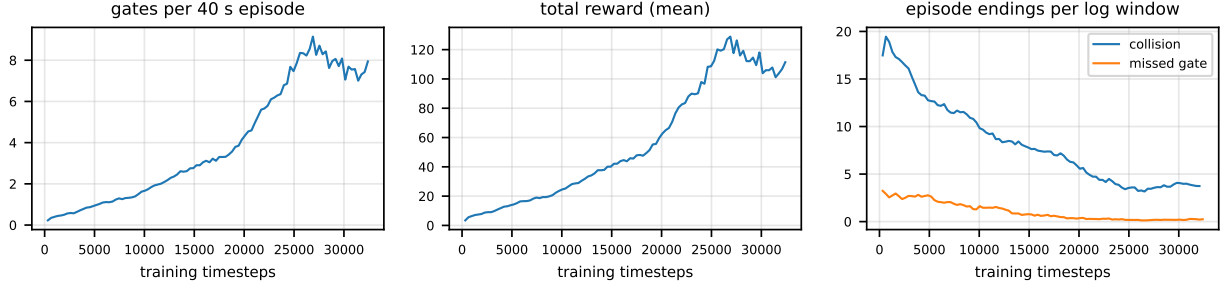


Figure 6: Training curves of the one-hour fine-tune.

Test on the corrected simulator	Gates per attempt	Crashes / 100 gates	$P(2 \text{ laps, no crash})$	Median lap (s)
No disturbances – Gene’s model	0.9	51.4	0 %	–
Expected disturbances – Gene’s model	0.3	110.3	0 %	–
Expected disturbances, race start – Gene’s model	0.6	70.3	0 %	–
Rougher disturbances – Gene’s model	0.1	302.7	0 %	–
No disturbances – 1-hour retrain	1.2	59.7	0 %	–
Expected disturbances – 1-hour retrain	6.3	11.2	7 %	11.01
Expected disturbances, race start – 1-hour retrain	4.7	11.8	6 %	10.95
Rougher disturbances – 1-hour retrain	0.3	134.2	0 %	–

Table 13: Gene’s model vs the 1-hour retrain on the corrected simulator: official course and real 74° camera (512 drones, 25s each). “Race start” means every attempt starts in front of official gate 1. “Expected” and “rougher” apply all disturbances of that realism level at once (Section 9). Gates per attempt counts every start-to-crash attempt; a crash rate above 100 means most attempts crash before passing a single gate.

- Clean playback from official gate 1, Gene’s model: 11×4 gates then collision; laps: no complete lap.
- Clean playback from official gate 1, the 1-hour retrain: 15×2 gates then collision; laps: no complete lap.

What the retraining test shows

- **Gene’s model does not survive the corrections.** On the corrected simulator (official course and real camera), it gets through only 0.1–0.9 gates per attempt: 51 to 303 crashes per 100 gates. In a clean playback run starting at official gate 1, it passes 4 gates and crashes, every time. With the real camera and course, expect it to crash within the first few gates.
- **One hour of further training helps a lot, but only under the conditions it trained on.** With the expected disturbances:
 - the retrained model gets through 6.3 gates per attempt, with 11 crashes per 100 gates. Gene’s model gets 0.3 and 110, so this is about 10× better;
 - when it completes a lap, the lap takes about 11s, slower than Gene’s 7.4s laps in his original simulator.
- **It is not yet reliable, and it only works in a narrow range.**
 - Only 6–7 % of runs would complete two laps.
 - It does worse with *no* disturbances at all, because it learned to expect delay and drag.
 - It still fails under rougher disturbances.
 - A clean playback run crashes after 2 gates.
- **Conclusion:** training on the corrected simulator is the right direction, but it needs several hours rather than one. It also needs the disturbances varied over wide ranges (camera tilt, delay, drag, detection noise), so that it copes with both calm and rough conditions, and every candidate must be checked with these tests before flying. Until a retrained model passes the Sunday go/no-go check on the real drone, Plan B is the plan for scoring.

9.4 How long proper retraining takes on bojro’s laptop

Job (RTX 4060 Laptop, WSL, 2048 drones, measured ≈ 9.3 –10 timesteps/s ≈ 20 k samples/s)	Time
One-hour feasibility fine-tune above (32.4k timesteps)	55–60 min
Standard 50k-timestep fine-tune (2048 drones)	≈ 1 h 25 min
Same samples as Gene’s 50k-timestep, 4096-drone runs (205M samples)	≈ 2 h 50 min
Gene’s model’s entire training history, from scratch (634M samples)	≈ 8 h 45 min

Table 14: Projected training times.

Constraints and caveats:

- **GPU memory:** 8 GB fits one Isaac job at a time (training uses 5.5 GB).
- **RAM:** WSL is capped at 7.5 GB and training uses 4.7 GB. 4096 drones would need a bigger WSL memory cap (a WSL restart) and might not fit in GPU memory.
- **CPU:** throughput is limited by the CPU/Python loop (the GPU sits near 52 %), so other CPU jobs slow training noticeably.
- **Power:** keep the laptop on AC and awake.

Gene’s RTX 4070 Laptop is of the same order per step.

10 Plan A and Plan B

	Plan A (existing policy)	Plan B (moderate, deterministic)
Brain	pq_speed_best PPO	state machine: fly to an approach point, face, align on the gate, creep, punch through
FC mode	ACRO (Jetson sends body rates)	ANGLE (FC self-levels; Jetson sends angles + throttle)
Speed	≈14 m/s mean, 25 m/s peaks; 7.4 s laps in Gene’s simulator	through gates at 2.5 m/s, up to 4 m/s between gates; ≈2:05 for two laps in simulation (“balanced” setting)
Needs	MSP bridge, real-image keypoints matching training, linear rate/throttle curves, gate counter; probably a retrain with real intrinsics and track	MSP bridge, gate detector tuned on real frames, hover throttle, pitch→speed calibration, FC mode change (ask first)
Risk	high (sim-to-real gap at speed)	low

Table 15: The two plans share the same foundation: MSP link, camera capture, logging, bench checks.

10.1 Plan B speed budget

- **Why not crawl:** too slow risks an unknown run-time limit (VQ capped runs at 8 min), eats 15-minute slots, drains the battery while hovering, and accumulates drift between gate sightings.
- **Why not faster:** above about 4 m/s, a look-and-servo controller runs out of turning radius and latency margin.

Tier	Cruise	Tightest turn (20° bank)	Stop distance	Latency travel (100 ms)	Lap	2-lap run
slow	1.5 m/s	0.6 m	0.4 m	15 cm	≈120 s	≈4.3 min
medium	2.5 m/s	1.8 m	1.2 m	25 cm	≈75 s	≈2.9 min
quick	3.5 m/s	3.4 m	2.3 m	35 cm	≈57 s	≈2.2 min
(too fast)	5.0 m/s	7.0 m	4.8 m	50 cm		

Table 16: Plan B speed limits: back-of-envelope physics for flying *through a gate* at each speed. Between gates the drone may go faster, because it slows down before each gate. In the simulation these became the settings “safe”, “balanced” and “fast” (Table 17; the simulation in Section 10.2 supersedes the lap estimates). Clearance inside the 1.5 m opening is about ±0.45 m for a roughly 0.6 m airframe (measure ours).

- **Speed control without a velocity sensor:** in ANGLE mode, speed settles where $v \approx g \tan(\text{pitch})/k$. Calibrate pitch→speed from one logged straight flight, close the loop on the gate’s apparent growth (range rate), and cap pitch at 12°.
- **Anti-stall rules:** alignment is time-boxed to 1.5 s; a lost gate gets one search sweep, after which the drone slows down rather than aborting; each leg has a time budget of 2× its expected duration.

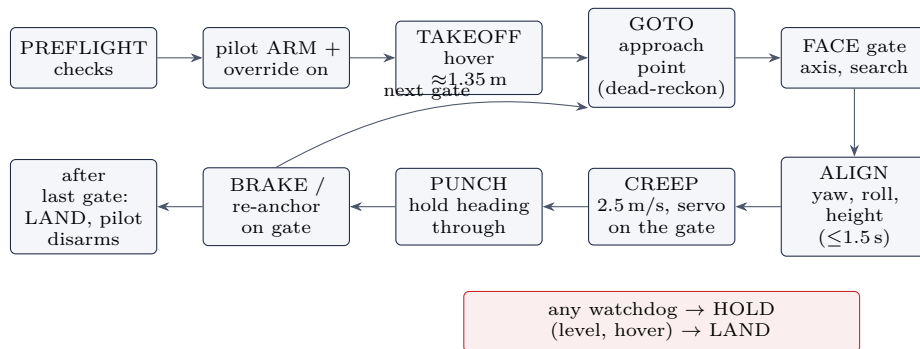


Figure 7: Plan B state machine. Special moves: at the G6 split-S, pass north of the gate and turn back east; at G9, top opening southbound, descend, then bottom opening northbound.

10.2 Plan B tested in simulation (16 Sep)

Plan B was implemented and tested in a purpose-built CPU simulator (`~/aigp/tools/planb_sim.py`), run in parallel while the GPU was busy with Isaac. It is lower fidelity than Isaac, but it models the things that decide whether Plan B can work.

What the simulator models:

- **Airframe:** a simplified drone (a point with mass) flying in Betaflight’s self-levelling ANGLE mode, with first-order attitude, yaw-rate and thrust lags; linear drag; per-run thrust-calibration error; random bumps (rougher conditions only).
- **Course:** the official layout, two laps plus the finishing pass through gate 1 (23 gate crossings; the double gate counts twice per lap), with every gate shifted per run away from the map.
- **Vision:** the real $74^\circ \times 46^\circ$ camera, tilted 10° up. Plan B flies slowly with little forward lean, so it uses a smaller tilt than the 20° in Gene’s simulator; the tilt is adjustable on the drone. Whenever the next gate is in view at 1.2–14 m, the camera gives a position and heading measurement relative to that gate. That measurement has noise that grows with distance, occasional dropouts, and a delay.
- **Between camera measurements:** the drone estimates its motion from its tilt, its thrust and an air-drag estimate (off by up to $\pm 10\%$ in expected conditions and $\pm 30\%$ in rougher ones). Height comes from the flight controller’s barometer (air-pressure altimeter), whose drift is corrected by the camera. Heading slowly drifts, and the tilt reading has a small error.
- **Scoring on the true state:** a pass requires the airframe (0.3 m half-span, 0.12 m half-height) to clear the 1.5 m opening. Touching any frame, the floor or the boundary, or missing a gate, ends the run.

The controller is the Plan B state machine:

- **Takeoff:** a staged throttle ramp.
- **Approach:** point the camera at the gate, and commit only when height and lateral offset are lined up. Commitment is time-boxed, with fallbacks at 1.5 s, 4 s and 8 s.
- **Pass:** fly along the gate axis.
- **G6 split-S:** bypass with the camera on G6, then stop on the far side with G6 in view.
- **G9 double gate:** climb, pass the top opening, descend and turn around, then pass the bottom opening.

Setting	gate speed	cruise speed	turnaround exit speed	ideal sensing finish / time	expected conditions finish / time	rougher conditions finish / time
safe	2.5	2.5	0.35	100 % / 2:17	100.0 % / 2:21	93.0 % / 2:34
balanced	2.5	4.0	0.8	100 % / 2:01	100.0 % / 2:05	88.4 % / 2:18
fast	3.0	5.0	0.8	100 % / 1:48	99.4 % / 1:52	79.4 % / 2:10

Table 17: Plan B, 500 simulated runs per cell (speeds in m/s; time is the median for a complete 2-lap run). In the 36-point sweep (Fig. 8), consistency collapses once the *gate* speed reaches 3.5 m/s.

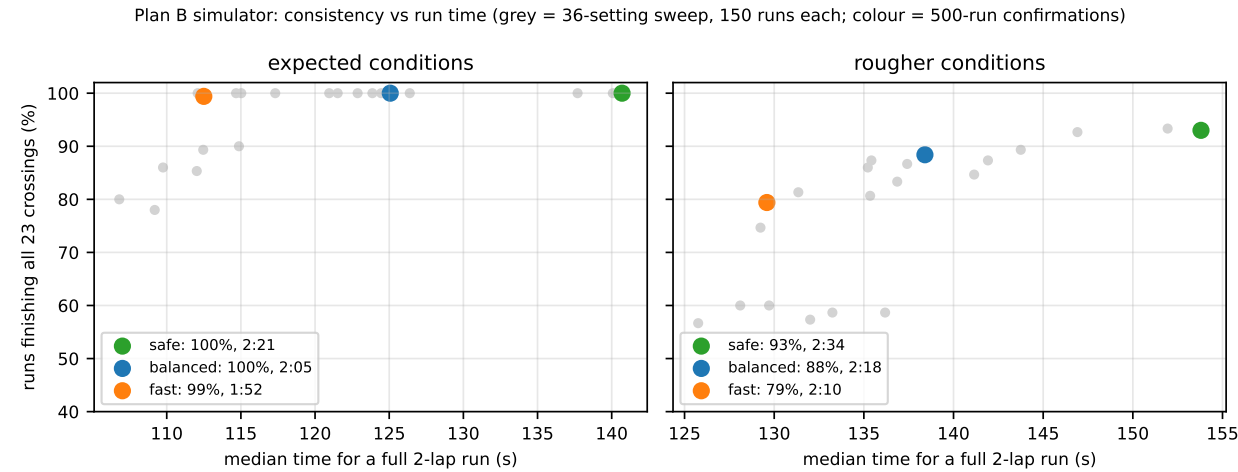


Figure 8: Consistency against run time for every swept Plan B setting.

Assumption changed (setting “balanced”, 300 runs each)	finish, expected	finish, rougher
as designed	100.0 %	87.3 %
no barometer (vision height only)	100.0 %	82.7 %
detector range 8 m	100.0 %	85.7 %
detector range 5 m	97.0 %	47.0 %
no back-of-gate detection	97.7 %	64.3 %
gate fixes at 10 Hz, +100 ms latency	99.7 %	69.0 %
camera tilt unknown by 2°	100.0 %	83.0 %
camera tilt unknown by 5°	92.3 %	47.0 %
all of: no baro, 8 m, no back view, 10 Hz, 2°	97.7 %	45.0 %

Table 18: How much Plan B depends on its assumptions.

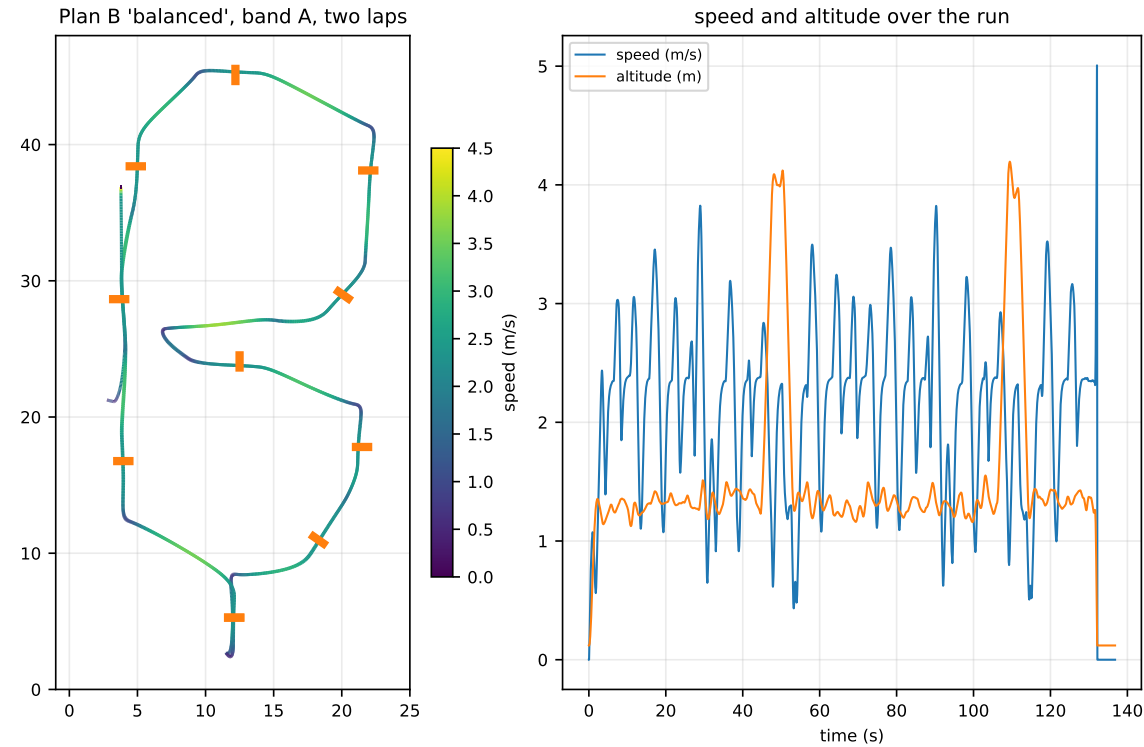


Figure 9: One simulated “balanced” run under expected conditions: split-S bypass around G6, climb to the upper opening of G9, two laps.

Plan B verdict

- **Recommended setting: “balanced”.** It finishes 100 % of runs under expected conditions and 88 % under rougher ones, in about **2:05** for two laps.
- **If on-site conditions look rough:** use “safe” (2:21, 93 % under rougher conditions).
- **Only after clean real runs:** try “fast” (1:52, 79 % under rougher conditions).
- **Robustness under expected conditions:** it still finishes 98 % of runs with *all* pessimistic assumptions at once.
- **A barometer is optional.** Vision height relative to each gate teaches the thrust bias.
- **The biggest single risk is camera tilt.** A 5° unknown tilt error drops finishing to 92 % (expected conditions) and 47 % (rougher). **Measure the tilt.**
- **Under rough conditions,** a short detector range (5 m), no back-of-gate detection and slow vision matter most.
- **What limits run time:** the two forced turnarounds and the alignment pauses, not the cruise speed.

What this simulation does not cover

- Real rotor aerodynamics near frames, and ground effect.
- Detector false positives (orange pylons, red signage).
- Wrong gate identity.
- A real gate detector’s accuracy. The test assumes a gate-relative pose with a few pixels of error.
- Whether ANGLE mode is permitted; ask the organizers.
- The airframe span; measure the real drone.

It shows that the Plan B *logic and speed budget* are sound. It does not replace testing in the cage.

11 Knowing which gate is which

11.1 The problem, and why it is not a perception problem

The policy is told which gate it is flying at. In the simulator that arrives for free. On the real course nothing publishes it, ten gates are identical 2.7 m squares, and **we asked the organizers whether we may mark them and the answer was no.**

Measured on our own course: **34 % of frames have two or more gates in view at once.** So “look at the gate and recognise it” is not available from appearance, ever.

The reframe that makes this tractable

Gate identity is not a perception problem. It is a **localisation** problem. If you know where you are, identity is a lookup: project all ten gates from the map and see which one you are pointed at. If you do not know where you are, no amount of staring at an identical square will tell you. Every published system reaches the same conclusion and none of them tries to recognise gates individually.

11.2 What the policy actually reads

Worth stating explicitly, because the argument below turns on it. Each frame is 51 numbers, and the policy reads **32 frames of history**, so 1632 inputs.

Index	Name	Meaning
0–15	<code>u0,v0 \dots u7,v7</code>	Eight gate corners as image positions, normalised by 640×360 . –1.0 in both when not seen
16–23	<code>vis0 \dots vis7</code>	Is that corner seen? 1.0 or 0.0
24–25	<code>roll, pitch</code>	Attitude from the flight controller, radians
26–28	<code>gx, gy, gz</code>	Gyro rates, body NED, clipped ± 8 rad/s
29–31	<code>vx, vy, vz</code>	Commanded body velocity, NED, clipped ± 20 m/s
32–49	<code>gate0 \dots gate17</code>	Which gate is next — one slot set to 1.0
50	<code>lap\frac</code>	Progress round the lap, index/17

Table 19: The 51-number observation frame. Verified against the training code element by element (Section 11.6).

The eight corners are **one gate only** — the one being flown at, not all ten. Corners 0–3 are the outer 2.7 m frame, clockwise from top-left; corners 4–7 are the inner 1.5 m opening in the same order. Having both rings is what encodes *range*: their true sizes are known, so apparent size gives distance.

Three things the policy is **not** given, which matter for what follows: no distance, no position or altitude or heading (roll and pitch but not yaw), and no sight of the *next* gate. Everything beyond the gate in view comes from the one-hot label and what the network memorised in training.

The consequence

Nothing in indices 0–31 distinguishes gate 3 from gate 8. All ten are identical squares. The **only** channel carrying course position is the one-hot in 32–49 — which is 576 of the 1632 numbers once the 32 history frames are counted, over a third of everything the network reads.

11.3 “The model will learn the order from simulation”

A reasonable-sounding proposal was put forward: label the gates in simulation, train on that, and the model will know the order on the real unlabelled gates.

The mechanical objection is that the label is an **input**, not an output. Training with it does not teach the network the order; it teaches the network to *depend on being told* the order. At run time something must still supply those 18 numbers, and a network cannot supply its own input.

But that is an argument, and arguments lose to measurements. So we measured it.

The experiment. Gene’s model, flown in *his* simulator on *his* track — his gate positions, his 90° camera, the 607.6 g drone — the conditions where it works well. 512 drones at once, 20 s each. Identical physics, course, camera and seed across all five runs. **The only thing changed is where the gate label comes from.**

Label source	Gates per crash	Crashes / 100 gates	Median lap	Laps completed
Correct, as trained	26.9	3.4	7.45 s	515
Stuck on the start gate	1.8	51.5	—	0
No label at all	1.1	85.6	—	0
Random label each frame	1.1	72.9	—	0
Off by one	1.0	80.7	—	0

Table 20: The gate label is load-bearing. Raw data in `~/aigp/eval/ctx/`; reproduce with `run_ctx.sh`.

What this settles

- **With the correct label the model is healthy:** 7.45 s laps, 16 m/s at the gate, 515 complete laps, about 27 gates between crashes. Exactly the performance it always had.
- **Without a label it completes zero laps.** Not one, across any of the four broken conditions. It crashes after roughly one gate.
- “Train with labels in simulation, then have nothing supplying them in reality” *is* the “no label at all” row.
- **Off-by-one is as bad as no label at all** (1.0 vs 1.1). This is not only an argument for having a gate counter — it is an argument for the counter being *right*. A counter that slips once is worth nothing.
- **A frozen counter is the least-bad failure** (1.8). So when confidence is lost, hold the last index rather than guess a new one. That is how ours is built.

The instinct behind the proposal is not wrong, and is a real published technique — but it requires *changing how we train*, not merely training with labels present. Either drop the input entirely and force the network to infer position from its 32 frames of history, or train a teacher with the privileged label and distil it into a student that sees only what the drone sees. Both work. Both have the same weakness: when the network’s internal guess is wrong there is no error signal, so it is confidently wrong. The approach below at least knows when it is lost.

11.4 What we build instead

The published systems — Swift^[ext], AlphaPilot^[ext] and the 2026 dual pose-graph work^[ext] — all share one shape: detect gate corners, solve a relative pose per gate, match against the *known map* in metric space, and fuse into a filter. Swift states it plainly: each observation is assigned to “the nearest gate in the known trajectory layout”.

Two things we do differently, both because we measured rather than copied.

No visual-inertial odometry. Every one of those systems leans on VIO. We have one forward camera and a 25 W budget. So we measured whether we need it: a gate is in view **75–79 %** of frames, the worst gap is 1.5 s at 5 m/s, and dead reckoning drifts **under a metre** across that gap — against gates that are 7.54 m apart. VIO would be solving a problem we do not have.

Throw away the PnP rotation. Planar PnP is ill-conditioned near head-on, which is where a racing drone spends its life. But we do not need its rotation: the flight controller already gives us attitude, far more accurately.

Corner noise	Range	Full 6-DOF PnP	PnP translation + FC attitude
2 px	8 m	2.12 m	0.09 m
5 px	8 m	3.15 m	0.29 m
10 px	12 m	6.81 m	1.38 m

Table 21: Discarding the ill-conditioned half of the PnP solution is worth 23× at the operating point. This is not from the literature; it fell out of the accuracy test.

11.5 The pipeline

1. **Detector** — finds a gate’s eight corners in the image. *Still to be built; needs real footage.*
2. **PnP** — corners plus the known 2.7 m/1.5 m geometry give range and bearing to the gate. Planar PnP returns two mirrored solutions; we pick by reprojection error and break ties with **gravity**, since gates stand vertically and we know which way is down.
3. **Position** — range and bearing, plus the flight controller’s attitude, plus the gate’s known map position, give *where we are*.

4. **Filter** — carries position through the gaps by integrating thrust and attitude, corrected by each fix. It also estimates **thrust scale**, which is what makes a barometer unnecessary — and the barometer is unusable anyway with the props spinning.
5. **Tracker** — knowing where we are, project all ten gates and see which one is ahead. Advance on a confirmed crossing. Fill the 18 one-hot slots.

In one line: *the gate looks this big and sits here in frame → so it is 8.3m away, 12° left → the map says that gate is at (12.19, 45.11) → so I am at (10.4, 37.2) → therefore this is gate 3 → set slot 35 to 1.0.*

11.6 Evidence

Check	Result
Observation frame vs training code	Exactly zero difference, 51 numbers $\times$ 200 cases
History stack vs training code	Exactly zero difference across 1632 numbers
Projection vs training code	3.9×10^{-6} px, zero visibility mismatches over 3200 corners
Policy in NumPy vs PyTorch	2.5×10^{-6} max error on the real checkpoint, 126 μ s per inference
Position, normal conditions	0.88 m median, 2.79 m p95 against 7.54 m gate separation
... with 20 % thrust error	0.88 m / 2.79 m (thrust scale learned 0.895 vs true 0.900)
... drag model 2 $\times$ wrong	0.80 m / 3.01 m
... 10 px corner noise	1.01 m / 2.38 m
... 7 m detector range	0.77 m / 2.29 m
... 12 m/s flight	0.77 m / 1.57 m
Dead reckoning with no fixes	88.9 m / 177 m — useless alone, which is the point
Gate counting, 17 conditions	22/22 crossings, zero errors (50 % dropout, 20 false positives/s, 100 ms latency, 1 m pose noise)

Table 22: Everything above is reproducible with `python run\_all.py` in `flight/`.

The one known limit: heading drift

The position fix is computed *using* our heading, so a wrong heading corrupts the fix directly and no filter can recover — the measurement itself is wrong. Measured tolerance:

Yaw drift	Median	p95	
0.05°s^{-1}	0.28 m	0.97 m	fine
0.5°s^{-1}	0.90 m	2.81 m	fine
1.0°s^{-1}	1.47 m	6.81 m	breaks

So it tolerates about 0.5°s^{-1} and fails near 1°s^{-1} . A calibrated Betaflight gyro should be an order of magnitude better than that, but this is now a specific number to **verify from the measurement flight** rather than assume.

12 Technical gap register

12.1 First, what Betaflight actually is

Several gaps below are “Betaflight things”, so here is the minimum needed to read them.

The flight controller, in plain terms

The **flight controller** is a small circuit board on the drone with a gyroscope on it, running firmware called **Betaflight**. Its job is narrow and fast: take four numbers — throttle, roll, pitch, yaw — several thousand times a second, and spin the four motors so the drone does what those numbers ask. It handles the violent, millisecond-scale balancing that keeps a quadcopter in the air. We do not have to write that, and we could not do it better.

Normally those four numbers come from a **radio receiver** wired to the flight controller, listening to a human pilot’s remote control.

MSP (MultiWii Serial Protocol) is simply a message format for talking to the flight controller down a serial wire. Betaflight speaks it. The feature we need is called **MSP override**: while a switch on the pilot’s remote is flipped on, Betaflight takes its four stick numbers from MSP messages — from our Jetson — instead of from the radio. Flip the switch off and the pilot has the drone back, immediately, in firmware. That is our kill path.

Two more terms that appear below:

- **ACRO and ANGLE** are flight modes. In **ACRO**, stick position means *rate of rotation*: hold the stick over and the drone keeps rolling; centre it and it stops rolling but stays tilted wherever it ended up. In **ANGLE**, stick position means *lean angle*: centre the stick and it returns to level by itself. Our trained policy outputs rotation rates, so it needs ACRO. Plan B wants ANGLE, because “hold still and hover” is then almost free.
- **Blackbox** is Betaflight’s built-in flight recorder. It logs throttle, gyro, motor outputs and battery voltage during a flight, and you download it over USB afterwards. Almost every measurement we need about the drone comes out of it.

12.2 Gap register

Effort is one person’s working time. “Blocks” names what cannot start until this is done.

ID	Gap	What it means, and why it matters	Effort	Blocks
A — Flight controller and the link to it (<i>much reduced: the organizers ship the protocol layer</i>)				
A0	Copy ~/target/ off a board	<code>scp -r dcl@192.168.55.1:~/target ./</code> Until somebody does this, nobody else can develop against the fake flight controller.	10 min	All adapter work
A1	Adapter, not protocol	MSP framing is provided. We write the layer that turns policy output into <code>RCTransmitter.set_control()</code> calls.	4 h	Everything autonomous
A2	Free the serial port	<code>sudo setup_jetson_uart.sh</code> -{}-apply, once per board. Linux runs a login console on that port by default. Skipping this looks exactly like a dead flight controller.	5 min	Any link at all
A3	Rate curve inversion	Still ours. Betaflight maps stick to rotation rate along a curve (<code>rc_rate</code> , <code>super_rate</code> , <code>expo</code>), gentle in the middle and aggressive at the ends. Our policy speaks radians per second. A linear approximation is wrong by tens of percent mid-range.	4 h	Accurate control
A4	Throttle curve inversion	Still ours. Same for throttle (<code>thr_mid</code> , <code>thr_expo</code>).	1 h	Height control
A5	Sign and axis mapping	Still ours, and the most dangerous item here. Isaac is forward-left-up, Betaflight is forward-right-down, plus channel order. A reversed sign flips the drone on takeoff. Verify one axis at a time, props off.	2 h	Any flight

ID	Gap	What it means, and why it matters	Effort	Blocks
A6	Watchdog: configure and verify	<code>RCTransmitter</code> already has one, in its own thread. Set the timeout, then prove it against <code>fake_fc.py</code> 's frozen mode.	1 h	Safety sign-off
A7	Loop rate mismatch	Organizers: attitude at 30–50 Hz is realistic, a hard real-time loop is not. Our policy runs at 60 Hz and eats gyro as input. Measure, then match the simulator to it.	2 h	Training validity
A8	Gyro units unverified	Sources conflict: raw counts at 16.4 per $^{\circ}\text{s}^{-1}$ versus already in $^{\circ}\text{s}^{-1}$. Verify empirically — the wrong choice is a silent $16\times$ error.	30 min	Sensor fusion
A9	Arming flow	<code>mcp_bench.py demo</code> clears an MSP arming lock, arms, then re-asserts it; Betaflight also blocks arming while an MSP client is connected. Read their code and copy the sequence.	2 h	Any real run
A10	Override mask: verify, do not assume	The mask = 11 finding (throttle not overridden) came from <i>another team's</i> dump. Since <code>demo</code> drives throttle successfully, ours may already be right. Check <code>diff all</code> on our drone.	10 min	Autonomous hover
A11	ANGLE mode not assigned	Flies ACRO; Plan B needs ANGLE. CLI-assignable — confirm it is permitted.	15 min	Plan B
A12	Blackbox not configured	Must be on and recording gyro debug data, or the measurement flight yields nothing.	15 min	All diagnostics
A13	Camera/IMU time sync	No hardware sync line exists on this carrier. Must be solved in software, and the exposure must be pinned first or auto-exposure moves the capture instant every frame.	4 h	Latency measurement
B — Onboard software				
B1	Capture with <i>real</i> timestamps	Viewers exist. But <code>cv2.VideoCapture</code> discards the hardware capture timestamp and gives arrival time with tens of ms of jitter. Use the GStreamer path where timing matters.	4 h	Perception
B1b	Image processor needs a display	Colour, auto-exposure and white balance run in NVIDIA's ISP, which needs a graphics context a plain SSH session lacks. Headless gives flat, un-exposed frames <i>by design</i> . Arrange a display context, or design for minimally-processed frames.	4 h	Usable images
B2	No framework on the board	Confirmed: OpenCV, NumPy, pyserial, python3-gi — no PyTorch. The NumPy policy runtime is now <i>mandatory</i> . The detector must go to ONNX via OpenCV's DNN module or TensorRT. Never <code>pip install opencv-python</code> : it silently breaks the camera pipeline.	3 h	Plan A
B3	No gate counter	The “which gate is next” input has no source on the real drone.	1 day	Plan A
B4	No takeoff routine	The policy was trained starting <i>in the air</i> near a gate. The real race starts on the floor. Something must take off, stabilise, and hand over.	4 h	Any real run

ID	Gap	What it means, and why it matters	Effort	Blocks
B5	Crash and abort logic is simulator-era	The existing logic auto-re-arms after a crash, which is a simulator behaviour and dangerous on a real drone.	3 h	Safety sign-off
B6	No onboard logging standard	Every flight needs a synchronised record of inputs, outputs and detections, or we cannot diagnose anything.	2 h	All learning
B7	Compute budget unverified	Detector plus policy plus capture, at 25 W, at the target frame rate. Unknown until measured.	2 h	Loop rate

C — Perception

C1	Detector has never seen a real gate	Existing detectors were tuned on simulator images. See Section 14.6.	2 days	Any autonomy
C2	Camera not calibrated	The spec states no calibration will be provided, and it is a screw-mount lens.	1 h	Perception accuracy
C3	Camera tilt unknown	It is adjustable, even in flight, so it is a choice we must make and then measure. A 5° error took crashes to 32 per 100 gates.	30 min	Perception accuracy
C4	Sensor mode may not exist	The carrier wires only two of the camera's four data lanes , capping resolution $\times$ framerate. <code>v4l2-ctl -{}-list-formats-ext</code> is the truth, not the datasheet. The 75° figure is quoted for 1920 $\times$ 1080 at 60 fps — if that mode is missing, our field of view is wrong before we start.	10 min	Everything perception

D — Simulator fidelity

D1	Thrust-to-weight is a guess	Hover throttle assumed 25.5 %. Worth 58 crashes per 100 gates if wrong by 10 %.	5 min	Training validity
D2	Inertia and rate response wrong	5-inch airframe values on an 8-inch drone; see the warning in the status section.	4 h	Training validity
D3	Zero latency modelled	The most damaging single factor found: 50 ms alone took crashes from 2 to 55 per 100 gates.	3 h	Training validity
D4	No drag model	Significant above 10 m/s.	3 h	Training validity
D5	Perfect vision assumed	The simulator hands the policy exact gate corners with no noise, no dropouts and never the wrong gate. This is the largest single difference between simulation and reality.	1 day	Transfer
D6	Course map: official table in hand	The official coordinates confirm 85 $\times$ 165 ft and our corrected track matches exactly. Still survey the as-built placement — a 7.5 % scale error alone took crashes from 2 to 15.5 per 100 gates.	2 h	Training validity
D7	Policy rate may be too fast	Simulator runs a 60 Hz policy; real gyro feedback may be 30–50 Hz. Set decimation to the measured rate <i>before</i> the long training run.	1 h	Training validity

E — Operations

E1	No compiler on the laptop	Downgraded. The Betaflight software-in-the-loop build is no longer needed — the organizers' <code>fake_fc.py</code> is pure Python and does the job better.	10 min	nothing now
----	---------------------------	---	--------	-------------

ID	Gap	What it means, and why it matters	Effort	Blocks
E2	Practice slots unknown	Flight time is rationed and scheduled. Everything must be planned around the slot list.	—	All planning
E3	Battery and propeller count unknown	The real limit on how many attempts we get.	—	Risk budget

Table 23: Technical gap register as of 17 September, after reading the organizers’ Orin NX guides. Nothing here is a criticism of the existing code — almost all of it was written for the virtual qualifier, which was a different race with a different interface. The A-series shrank considerably once it became clear the organizers ship the protocol layer themselves.

13 The plan, organised as parallel workstreams

The mistake to avoid is queueing everything behind the flight-controller link. Only one workstream actually depends on it. Three others need no software at all and should start the same hour.

Stream	Name	Content	Waits for
WS-1	Link and control	MSP implementation, curve inversion, watchdog, bench validation (gaps A1–A12)	Nothing. This is the critical path
WS-2	Vehicle characterisation	Weigh, balance, measure; the piloted measurement flight; hover thrust, rate response, drag, battery sag (D1–D4)	Nothing — a human pilot and a blackbox log need no code
WS-3	Perception	Camera bring-up, calibration, data capture, labelling, detector, gate counter (B1, B3, C1–C4, D5)	Camera only
WS-4	Course survey	Tape-measure every gate from a fixed datum; reconcile with both documents; build the map (D6)	Nothing — hall access and a tape measure
WS-5	Simulator and training	Fold measurements in; cloud training with randomised unknowns; evaluate with the stress harness	Nothing — starts with wide randomisation
WS-6	Integration and flight test	The bring-up ladder, in practice slots	WS-1, plus WS-3 and WS-4 for gate work
WS-7	Plan B	The slow stop-and-line-up controller	WS-1, WS-3
WS-0	Operations	Slots, batteries, organizer questions, installs, logistics	Nothing

Table 24: Eight workstreams. Five of the eight can start immediately and in parallel.

How to staff this

Put your strongest systems person on **WS-1** alone, with a second person reviewing the sign and axis conventions — a reversed sign there flips the drone on takeoff, and it is the single most common first-flight failure in this whole field.

WS-2 needs the pilot plus one engineer, and it is the cheapest high-value work available: one battery, one log, and the simulator stops guessing.

WS-4 is two people and ninety minutes, and it retires a risk worth 15 crashes per 100 gates. Do it on day one and do it again before the scored days.

WS-3 is the longest pole after WS-1 and it is mostly patient work — capture, label, train, measure. Start the capture the hour the camera produces a frame.

14 Field procedures

14.1 P1 — Camera calibration

1. Confirm the sensor mode first (gap C4). The 75° figure applies to 1920×1080 at 60 fps. If our pipeline captures another mode, the field of view is different before we start.
2. Print a checkerboard or ChArUco target and **glue it to something rigid and flat**. A curled printout produces a confident, wrong calibration.

3. Capture 30–50 views at the race exposure and gain settings: target near and far, tilted in both axes, in all four corners of the frame.
4. Solve for focal length, principal point and distortion (OpenCV). Expect a reprojection error below about half a pixel; if it is worse, the target moved or the board is not flat.
5. **Sanity check independently:** tape measure flat against a wall, camera exactly 2.00 m away, and confirm the width spanned matches the focal length you solved for.
6. Record the result with the date and the specific drone. The four airframes may not match.

14.2 P2 — Camera tilt

1. Set the drone on a surface you have confirmed is level with a spirit level.
2. Measure the camera mount angle with a phone inclinometer held against the lens face or the bracket. Repeat three times and take the median; this is easy to get wrong by a couple of degrees.
3. Cross-check optically: place a target at a known height at a known distance and confirm it lands where the calibrated model predicts.
4. Tilt is adjustable, so **decide** it: a lower tilt (around 10°) suits Plan B’s slow, near-level flight; a higher tilt (around 20°) suits fast forward-leaning flight. Choose, set, measure, and write it down. Then use that number in the simulator.

14.3 P3 — Vehicle diagnostics: one battery, one log

This is the highest-value thirty minutes of the week. Fly it with a human pilot, with blackbox logging switched on, in this order. It needs no autonomy software at all.

#	Manoeuvre	What it yields
1	Armed on the ground, 10 s idle	Gyro noise floor; vibration spectrum; motor idle value
2	Stable hover at 1.5 m for 20 s	Hover throttle — gap D1, the most valuable number available. Also a ground-effect reference
3	Three sharp rolls, three pitches, three yaws, settling between each	Rate response: rise time, overshoot, peak angular acceleration, per axis — gap D2. This is what replaces guessing the inertia
4	Full-throttle vertical climb, 1.5 s, then recover	Maximum thrust and true thrust-to-weight
5	Hold about 20° of pitch down a straight until the speed stops rising	Drag — gap D4. Drag coefficient $\approx mg \tan \theta / v^2$
6	Hover again for 20 s near the end of the pack	Battery sag: how much hover throttle rises as the pack empties
7	Land, disarm, download the log, and save a <code>diff all</code> from the CLI	The settings dump. Do not skip this; it is the thing that will be missing at 2 a.m.

Repeat manoeuvre 2 on every airframe you might fly. Hover throttle is the number the whole simulator hangs from, and four supposedly identical drones are rarely identical.

14.4 P4 — Latency

Latency is the most damaging single factor in the durability study, and it is also the one people estimate instead of measuring. Measure it.

1. Synchronise the clocks first (gap A12), or the answer is meaningless.
2. Put a bright LED in the camera’s view, wired so the Jetson can switch it.
3. Flash it, and record: the time the Jetson commanded the flash; the timestamp of the first frame showing it lit; the time the detector produced an output; the time the command went out over MSP; and the time the gyro first responded.
4. That chain gives you each stage separately, which matters — knowing the total is 90 ms is useful, knowing that 60 ms of it is the detector tells you what to fix.
5. Feed the measured total, and its variation, into the simulator as a randomised band.

14.5 P5 — Course survey

1. Choose a datum: a permanent corner as the origin, plus a second fixed feature to define the direction of the x axis. Mark both with tape and photograph them.
2. For each gate, measure the **base of both uprights** from the datum. Two points give you the centre *and* the heading in one operation, which is far more reliable than measuring an angle directly.
3. Record the bottom-bar height for one gate of each type, and confirm the double gate's upper opening centre.
4. Note what the drawing does not show: pylons, netting, lighting rigs, anything above 4 m.
5. Convert to the simulator's frame and difference against *both* the organizer table and the drawing. Reconcile every gate that differs by more than 0.25 m before training on it.
6. Re-measure every morning and before every scored run. Ask what tolerance gates are re-placed to; that number becomes the gate-jitter range in training.

14.6 P6 — Gate detection and data labelling

This deserves its own treatment, because it is the largest difference between our simulator and reality, and it is the workstream most likely to be underestimated.

Why this is hard, when the simulator makes it look easy

In the simulator, the policy is handed the eight corners of the gate as exact pixel coordinates, computed by projecting the gate's known geometry through a perfect pinhole camera. There is no noise. Corners are never missed. It is never the wrong gate. Visibility is known exactly.

On the real drone, every one of those is false. You have a compressed image of an orange-and-white frame against a cluttered hall, with motion blur, a rolling shutter that skews fast-moving objects, changing light, other gates in frame, orange pylons that look like gate material, and a detector that will sometimes return nothing and occasionally return something confidently wrong.

Closing this gap has two halves, and the second is the one teams skip: make the detector as good as is reasonable, *and* make the policy tolerant of a realistic detector by reproducing the detector's *measured* error statistics inside the simulator. Perfect detection is not achievable; a policy that degrades gracefully when detection is imperfect is.

Stage 0 — classical detector, first hour. Tune the existing colour-and-geometry detector on real frames — it lives in `vision/` as `snake\_gate\_detector`, and `tools/hsv\_tuner` adjusts its colour thresholds. No data, no training, no installation. It will be noisier and shorter-ranged than a learned detector, but it is available immediately and it is the fallback that cannot fail to be ready. Plan B tolerates a noisy detector far better than Plan A does.

Stage 1 — capture, starting the hour the camera works. Data capture is the bottleneck, so begin it before you need it. Capture must cover:

- distances from about 1 m to 15 m, and all approach angles including oblique;
- *every* gate on the course — the backgrounds differ, and that is what a detector learns;
- gates partially cut off by the frame edge, and two gates overlapping;
- the double gate, from below and from the upper approach;
- backlit and shadowed conditions, and whatever the lighting does across a day;
- **negatives:** frames with no gate, and frames containing confusers — orange pylons, red signage, other drones, people;
- **motion blur**, which means recording every manual pilot flight. Hand-carried footage is easy to collect but is missing exactly the degradation that matters in flight.

Stage 2 — labelling, done cheaply. Labelling is where teams lose a day. Three techniques cut the work by roughly five to ten times:

1. **Auto-label, then correct.** Run the classical detector over all footage and keep its output as draft labels. Correcting a drawn box is far faster than drawing one.
2. **Geometric auto-labelling — the big win.** We know the gate’s exact geometry (2700 mm outer, 1500 mm inner) and, after the survey, exactly where each gate is. If the camera pose for a frame is known, the eight corners project into the image exactly — the label is free and pixel-perfect. Bootstrap it: hand-label around 40 frames, solve the camera pose from those by PnP, then track along the video and re-project labels for every frame in between. This is how an afternoon produces thousands of labelled frames instead of a few hundred.
3. **Label what the policy actually needs.** The observation wants eight keypoints plus visibility flags — not bounding boxes. Keypoint labelling is slow. A hybrid is cheaper: let a learned model find the gate as a box, then fit the quadrilateral inside it with classical line fitting. Much less labelling, and `vision/yolo\_hybrid\_gate\_detector.py` is already shaped this way.

For a single, visually distinctive object class, fine-tuning a pretrained detector typically needs **300–800 well-chosen labelled frames**, not tens of thousands. Spend the effort on coverage and on the hard cases, not on volume.

Stage 3 — train and deploy. Fine-tune on the laptop GPU; it is a small model and takes minutes. Then build the accelerated engine **on the Jetson itself** — these are specific to the device and software version and cannot be prepared in advance. Budget real time for this at the venue.

Stage 4 — measure the detector, because this is what feeds the simulator. The useful metrics are not the ones detector papers report. Measure:

- detection rate against distance — what *is* our reliable range?
- corner pixel error against distance and viewing angle, as a median and a 95th percentile;
- false positives per frame, and specifically what triggers them;
- how often the identity is wrong;
- throughput and latency at 25 W.

Get ground truth by placing the drone at tape-measured positions relative to a gate: those frames have exactly known corner positions, so the error can be measured rather than estimated. `tools/eval\_detectors.py`, `tools/eval\_gate\_pose.py` and `tools/vision\_rate.py` exist for this.

Stage 5 — close the loop. Feed those measurements back as the simulator’s vision model: pixel noise as a function of distance, dropout rate, maximum range, frame rate, latency. Then retrain. This is the step that converts a detector measurement into a policy that survives a real detector, and it is the reason Stage 4 is not optional.

The gate counter. Separately from detection: maintain the expected gate number, predict where that gate should appear using the surveyed map, and accept a detection only if it lands near the prediction. Advance the count on a confirmed pass, with a short lockout so one crossing cannot count twice. **Encode explicitly that the double gate counts twice per lap** — one physical gate carrying two different numbers is where a counter desynchronises. If the expected gate is not seen for a while, widen the search, then fall back to a hold; never silently re-label, and log every re-association. Test all of this offline by replaying a hand-carried lap before it ever flies.

15 Schedule and decision gates

Day	Goal	Exit criterion
Thu 17	We can command the drone, and we know the course	Bench test with propellers off passes on all four axes; piloted measurement flight logged; hover throttle known; course surveyed; overnight training launched
Fri 18	We can see and count gates	Detector working on real gates; gate counter verified offline against recorded footage; autonomous hover; station-keeping in front of one gate
Sat 19	A complete lap, by any means	One clean Plan B lap. This puts a floor under the score. Plan A begins with the speed limiter at minimum
Sun 20	Rehearsal and freeze	Both plans rehearsed under run conditions; gates re-measured; configuration frozen; Plan A go/no-go
Mon 21	Scored	First slot: most reliable configuration, flown slowly, to bank gates. Then step up
Tue 22	Scored	Improve on the banked score. Never risk the drone before a valid run exists

Plan A go/no-go, Sunday. All three must hold:

1. it has flown at least three gates autonomously on the real drone, more than once;
2. its observations have been checked against the real camera and the real flight-controller curves;
3. its failure mode is a safe hold, not a flyaway.

The scoring rule that should drive every decision

Ranking is by **maximum number of gates passed**; time is only the tie-break; and it is the best run across the final two days. A slow run that completes therefore beats a fast run that ends at gate four. Bank a score first. Push afterwards. Do not let the last day be the first time we have completed a run.

16 Questions still open for the organizers

The specification answered several questions that Revision 1 listed. Answered: the interface is MSP over UART, with demo software provided; Betaflight CLI access covers rates, PIDs and filters but not reflashing; camera tilt is adjustable in flight; exposure and gain are controllable; no calibration matrices are provided; a valid run is two laps with no human pilot intervention; manual override always supersedes autonomy; practice is in scheduled slots.

Still open:

1. May we assign an ANGLE mode switch, and change `mcp\override\_channels\_mask`, the rate curve and the throttle curve?
2. May we place tape or markers on the gates to identify them? **A yes largely removes the gate-counting problem**, so ask early.
3. Who arms the drone for a scored run? Does arming and enabling override before the start line count as intervention?
4. Is there a maximum run duration? How many scored attempts per team per day?
5. Are gates re-placed between runs, and to what tolerance?
6. Are gates counted only in order — does skipping one stop the count? Is a run that crashes after N gates scored as N ?
7. Which camera sensor mode is the 75° figure quoted for, and can we change modes?
8. Are all four drones on identical firmware and configuration? Is a barometer fitted?
9. Where do we get the camera and serial demo applications?
10. Exact as-built gate heights, and the upper opening height of the double gate.

A Laptop environment (installed 16 Sep)

- **Hardware:** RTX 4060 Laptop (8 GB), 15 GB RAM (WSL sees 7.6 GB), 16 cores, WSL Ubuntu 26.04, no passwordless sudo.
- **Missing:** no compiler at all — `gcc`, `g++`, `make`, `cmake` and `ffmpeg` are absent, which blocks building the Betaflight software-in-the-loop simulator. `venv-client` has no `pip`. One command fixes both: `sudo apt install -y build-essential cmake ffmpeg poppler-utils`, then `python -m ensurepip`.
- **Repo:** `~/Downloads/ai-grand-prix/AI_GP`, branch `isaacsim`, read-only for us.
- **Flight client:** `~/aigp/venv-client` (Python 3.12, CUDA torch, ultralytics, OpenCV, pymavlink, pytest, tensorboard). The client test suite: 441 pass, 79 fail, 1 skip; the failures are in retired VQ code or are minor drift.
- **Isaac:** `source ~/aigp/isaac_env.sh`. Isaac Sim 4.5, Isaac Lab 2.1 (`~/aigp/IsaacLab`), skrl 1.4.2, torch 2.5.1+cu121.
 - Pins: `numpy<2`, `transformers<4.50`, `warp-lang<1.8`, `gymnasium<1.2`.
 - System libraries were unpacked from `.deb` files into `~/aigp/syslibs`.
- **Scratch copies:**
 - `~/aigp/idr_smoke`: original code.
 - `~/aigp/idr_A`, `idr_B`: bug A/B test.
 - `~/aigp/idr_exp`: bug fix, used for all experiments.
- **Tools and results:** `~/aigp/tools/eval_policy.py`, `stress_eval.py`, `make_figs.py`; results in `~/aigp/eval/`.

B Sources

- VADR-TS-005 PQ Technical Specification, issue 00.02 (2026-09-03); AI Grand Prix welcome packet.
- `theaigp.com` (home/FAQ, previous updates, official rules); public PQ spec TS-004; VQ2 spec TS-003.
- Organizer gate-coordinate table “Drone_Race_Track_Gate_Coordinates_with_doublegate_5800.pdf”, as published in another team’s repository.
- `github.com/JCorbin406/AIGP-Physical-Qualifier` (“AI of Sauron”): FC dump, MSP notes, camera calibration, on-site confirmations. No license, so facts are used and no code.
- Betaflight 4.4.3 source, `src/main/rx/msp_override.c`, `src/main/rx/msp.c`.
- `kousheekc/isaac_drone_racer` (BSD-3); Isaac Lab 2.1; skrl 1.4.2.
- Kaufmann et al., “Champion-level drone racing using deep reinforcement learning”, *Nature* 2023; Xing et al., CoRL 2024 (imitation plus RL for vision-based racing).